Video MP4 narrado a pa...
TXT 13.08KB

esto cuando puede utlizar de memoria cuando se ejecuta.

Consumo de memoria al ejecutar el generador de video

Analizando el código, el consumo de memoria **no es constante**: crece y se acumula durante toda la ejecución porque el script mantiene **todos los segmentos MP4 intermedios vivos al mismo tiempo**. Te lo desgloso:

1. Puntos donde se guarda en memoria (RAM)

a) `functools.lru_cache` en `_fuente()`

```python
@functools.lru_cache(maxsize=32)
def _fuente(tamano: int, negrita: bool = False) -> ImageFont.FreeTypeFont:
```

- Guarda **hasta 32 objetos** `FreeTypeFont` en RAM.
- Cada fuente cargada ocupa típicamente **1–3 MB** según el tamaño.
- Techo aproximado: **~30–90 MB** si se usan muchos tamaños distintos (los `_texto_ajustado` prueban 5–6 tamaños por texto, y hay dos modos: negrita y normal).

b) `_texto_ajustado()` — variable `lineas`

Aquí hay un **bug potencial de memoria/estado**:

```python
for tamano in tamanos:
    ...
    if len(lineas) * interlineado <= alto:
        return fuente, lineas, interlineado
# si ningún tamaño cabe:
maximo = max(1, alto // interlineado)
lineas = lineas[:maximo]
```

Si ningún tamaño cabe, se usa la variable `lineas` e `interlineado` **del último iteración** del `for` (funciona por suerte, pero no es limpio). No es fuga, pero es frágil.

c) `narrar([...])` — aquí está el mayor pico de RAM

```python
audios, motor = narrar([(texto, carpeta / nombre) for nombre, _, texto, _ in partes], voz)
```

Según el comentario:

> *"toda la narración de una vez: con Gemini se piden varios fragmentos a la vez"*

Esto significa que **se generan TODOS los audios en paralelo** (una petición por escena). Si el guion tiene, digamos, 30 escenas:

- 30 peticiones concurrentes a Gemini TTS.
- Cada respuesta de audio se materializa en memoria antes de escribirse a disco.
- **Pico estimado: cientos de MB** si el guion es largo.

Este es el punto más crítico y no está limitado por ningún semáforo ni `max_workers`.

d) `partes` y `segmentos`

- `partes`: lista de tuplas con closures (`lambda`). Cada `lambda` captura `i`, `t`, `n` → ligero, **KBs**.
- `segmentos`: lista de `Path`. Tras escribirse a disco, los MP4 **no** quedan en RAM.

e) `_segmento()` — pico puntual por invocación

```python
"-c:v", "libx264", "-tune", "stillimage", "-preset", "veryfast",
"-threads", str(settings.video_hilos),
"-x264-params", "rc-lookahead=5",
```

Buen detalle: se limita a 1 hilo (`settings.video_hilos`) y `rc-lookahead=5`. Eso mantiene el pico de x264 bajo (~50–150 MB por instancia). **Pero se ejecuta en serie**, un segmento por vez, así que no hay multiplicación.

f) `Image.new("RGB", (1280, 720))` en `_portada`, `_cierre`, `_diapositiva`

- Cada imagen RGB 1280×720 ≈ **2.7 MB**.
- Se crean, se dibujan y se guardan con `.save(destino)`, luego quedan elegibles para GC.
- Pico real: **~3–6 MB** por diapositiva, no acumulativo.

2. Puntos donde se guarda en disco (no RAM)

Archivo	Tamaño típico	Cuándo se libera
`000.png … 999.png`	~100–300 KB c/u	Al salir del `TemporaryDirectory`
`000.mp4 … 999.mp4`	**~1–5 MB c/u**	Al salir del `TemporaryDirectory`
`segmentos.txt`	KB	Idem
`video.mp4` (final)	Decenas de MB	Se mueve a `destino`

⚠️ **Aunque no sea RAM, el `TemporaryDirectory` acumula todos los MP4 intermedios en disco hasta que termina todo el `with`.** Para un guion de 40 escenas con audios de 20 s cada uno, son fácilmente **100–200 MB en `/tmp`**.

3. Resumen del comportamiento de memoria

Fase	RAM aproximada	Notas
Carga de fuentes (lru_cache)	30–90 MB	Acumulativo durante toda la ejecución
Render de diapositivas	3–6 MB por diapositiva	Se libera tras `.save()`
`narrar()` **masivo**	**Pico alto: 100–500+ MB**	**Punto más crítico**
Segmentos ffmpeg	50–150 MB por invocación (1 hilo)	Serial, no acumulativo
Concat final	Bajo (usa `-c copy`)	Solo I/O
Disco (`/tmp`)	~100–200 MB	Acumulativo hasta fin del `with`

4. Recomendaciones si te preocupa la memoria

1. **Limitar la concurrencia de** `narrar()` con un `asyncio.Semaphore` o `ThreadPoolExecutor(max_workers=2)`. Es el mayor consumidor.
2. **Bajar** `lru_cache(maxsize=32)` a `maxsize=8` (o quitar el `functools.lru_cache` y cachear solo por `(tamano, negrita)` en un dict con `WeakValueDictionary`).
3. **Narrar y segmentar por escena** (streaming), en vez de narrar todas primero y luego segmentar. Sacrifica paralelismo pero acota RAM.
4. **El** `TemporaryDirectory` **ya libera bien** al salir del `with`; solo preocúpate si el disco del sistema es pequeño.
5. `_texto_ajustado`: inicializa `lineas` e `interlineado` antes del `for` para evitar depender del último valor del bucle (más robustez que memoria).

En una máquina modesta (2 GB RAM) el proceso **puede ir justo o fallar** con guiones largos, casi siempre por el pico de `narrar()`, no por Pillow ni por ffmpeg.

para una maquina free de onrender creo que son 512mg. que se puede ajustar para que funcione.

Encontrados 18 resultados web

Con 512 MB de RAM en el plan gratuito de Render , el código tal como está **casi seguro fallará** para guiones con más de unas pocas escenas. El culpable principal es `narrar()` haciendo todas las peticiones TTS en paralelo, pero también hay ajustes importantes en ffmpeg y Pillow. 6 16

Aquí tienes los cambios concretos que puedes aplicar:

🔴 Crítico: Limitar la concurrencia de `narrar()`

El comentario del código dice *"con Gemini se piden varios fragmentos a la vez"*. Con 10 escenas, son 10 peticiones concurrentes, cada una materializando el audio en memoria antes de escribir a disco.

Solución: Modifica `narrar()` (en `exports/narracion.py`) para procesar en lotes pequeños:

```python
# En lugar de lanzar todas las peticiones a la vez:
import asyncio

async def _narrar_con_limite(textos, voz, max_concurrent=2):
    semaforo = asyncio.Semaphore(max_concurrent)
    async def _una_por_una(texto, ruta):
        async with semaforo:
            return await narrar_individual(texto, ruta, voz)
    return await asyncio.gather(*[_una_por_una(t, r) for t, r in textos])
```

El límite `max_concurrent=2` es el máximo realista para 512 MB. Si Gemini TTS permite streaming a disco, úsalo.

🟠 Importante: Ajustar parámetros de ffmpeg en `_segmento()`

El encoding x264 consume memoria proporcional al `rc-lookahead` y a la resolución. Tu código ya tiene buenos ajustes (`rc-lookahead=5` , `threads=1`), pero puedes ir más lejos : 10 18

```python
"-c:v", "libx264",
"-tune", "stillimage",
"-preset", "ultrafast",  # veryfast → ultrafast: menos memoria en RC
"-threads", "1",
"-x264-params", "rc-lookahead=2:sync-lookahead=0",  # 5 → 2
"-pix_fmt", "yuv420p",   # Ya lo tienes, reduce de 3 a 1.5 bytes/pixel
"-r", "15",              # 25 → 15 fps (opcional, reduce carga)
```

Nota: `-r 15` hará que el video se vea ligeramente menos fluido, pero para diapositivas estáticas es imperceptible y ahorra memoria y CPU.

🟡 Moderado: Reducir el uso de Pillow

a) Bajar el `lru_cache` de fuentes

```python
@functools.lru_cache(maxsize=8)  # 32 → 8
def _fuente(tamano: int, negrita: bool = False):
```

Con 8 posiciones en caché liberas ~70 MB de RAM.

b) Reducir resolución de trabajo o usar JPEG

Si aún va justo, puedes renderizar a 960×540 en lugar de 1280×720. La memoria de cada imagen cae de ~2.7 MB a ~1.5 MB, y ffmpeg puede escalar al unir.

🟢 Procesamiento secuencial por escena (cambio estructural)

El `TemporaryDirectory` acumula todos los MP4 intermedios hasta el final. Para 512 MB, lo más seguro es **narrar y segmentar una escena a la vez**, en lugar de narrar todo primero:

```python
# Pseudocódigo del cambio en generar_video:
segmentos = []
for i, escena in enumerate(guion.escenas, 1):
    texto = _texto_hablado(i, total, escena.narracion)
    # Narrar SOLO esta escena
    audio = narrar_individual(texto, carpeta / f"{i:03d}")
    # Renderizar imagen
    imagen = carpeta / f"{i:03d}.png"
    _diapositiva(i, total, titulo, escena.narracion, imagen)
    # Segmentar
    segmento = carpeta / f"{i:03d}.mp4"
    _segmento(ffmpeg, imagen, audio, escena.duracion_seg, segmento)
    segmentos.append(segmento)
```

Esto sacrifica paralelismo (tardará más), pero acota la RAM a lo que consume **una escena a la vez**.

📋 Resumen de prioridades

Prioridad	Cambio	RAM liberada estimada
🔴 **1**	Limitar `narrar()` a 1-2 concurrentes	200-500 MB
🟠 **2**	`rc-lookahead=2`, `sync-lookahead=0`	50-100 MB por segmento
🟡 **3**	`lru_cache(maxsize=8)`	~70 MB
🟢 **4**	Procesamiento secuencial (no acumular `partes`)	~100 MB de pico
🔵 **5**	Bajar resolución a 960×540	~1 MB por imagen

Con **solo el cambio #1** ya podrías pasar de "imposible" a "quizás funcione para 5-10 escenas". Con **#1 + #2 + #3** deberías poder manejar guiones de tamaño moderado.

18 páginas web

Narración en voz alta pa...
TXT 18.74KB

este archivo maneja, lo que comenta.

Análisis del módulo `narracion.py`

El archivo hace **más** de lo que su docstring sugiere. La descripción dice *"se piden varios a la vez"* (refiriéndose a voces Gemini en paralelo), pero el código actual implementa una estrategia muy distinta y **mucho más eficiente en memoria**. Vamos por partes.

1. Lo que el docstring promete vs. lo que el código hace

Promesa del docstring	Realidad en el código
"Cada fragmento es una petición; se piden varios a la vez"	❌ Falso. `_narrar_gemini()` **agrupa todos los fragmentos en UNA sola petición** (hasta 4000 caracteres por petición, luego divide en bloques secuenciales). No hay paralelismo.
"Si un fragmento falla con Gemini, TODA la narración se rehace con el sistema"	✅ Cierto. `narrar()` captura la excepción y rehace todo con `sintetizar()`.
"`say` en macOS o `espeak-ng` en Linux"	✅ Cierto, y con lógica de selección de voz por acento y calidad.

El docstring está desactualizado. El comentario real está en el propio código:

> *"El plan gratuito de Gemini TTS admite muy pocas solicitudes por minuto, así que toda la narración de un video va en UNA solicitud"*

Esto es **excelente para tu problema de 512 MB en Render**: significa que el pico de memoria de la narración ya no es "N peticiones concurrentes", sino **1 petición + su respuesta**.

2. Uso real de memoria por componente

a) `_narrar_gemini()` — el punto crítico real

```python
_MAX_CARACTERES_POR_PETICION = 4000
```

- Cada bloque de hasta 4000 caracteres produce un PCM que se **materializa completo en RAM** como `bytes` antes de escribirse a `.wav`.
- Gemini TTS devuelve **PCM 16-bit mono a 24 kHz = 48 KB/segundo**.
- Una lectura de 4000 caracteres a ~150 ppm tarda ~2.5 min ≈ 150 s.
- **Pico por bloque: ~7 MB de PCM**, más el WAV temporal (mismo orden).
- Si hay 30 escenas y caben en 1 solo bloque → **~7 MB pico**. Si no caben, los bloques se procesan **secuencialmente** (`for k, bloque in enumerate(bloques)`), así que el pico no se multiplica.

✅ **Esto ya es compatible con 512 MB.**

b) `_silencios()` y `cortes_por_silencio()`

- `_silencios()` lanza ffmpeg con `silencedetect` y parsea `stderr` con regex. No carga audio a RAM, solo lee la salida de texto.
- `cortes_por_silencio()` opera sobre listas de floats. Trivial.

c) El bucle de corte por fragmento

```python
```

```
for (_, destino), desde, hasta in zip(bloque, limites, limites[1:]):
    _ejecutar([ffmpeg, ..., "-ss", f"{desde:.3f}", "-to", f"{hasta:.3f}", ...])
```

- **Serial**, un ffmpeg a la vez.
- Cada ffmpeg lee el WAV completo desde disco (`-ss` y `-to` con `-i` antes implican decodificar desde el inicio).
- **Pico por invocación: ~10–20 MB** (buffer de ffmpeg + WAV).

d) `sintetizar()` con voces del sistema

- Escribe el texto a un `.txt` (evita pasar texto como argumento).
- `say` / `espeak` escriben directo a archivo.
- **Pico: despreciable** (el texto más largo son unos KB).

e) Caches

```
python

@functools.lru_cache(maxsize=1)
def _voces_macos()  # 1 entrada, lista de tuplas: KB

@functools.lru_cache(maxsize=4)
def _voz_macos(tipo)  # 4 entradas: KB
```

Sin impacto en memoria.

3. Comparado con `video.py`, este módulo está bien

Aspecto	`video.py`	`narracion.py`
Concurrencia TTS	❌ Asumida alta	✅ 1 bloque secuencial
Pico de audio en RAM	Alto (N respuestas en vuelo)	✅ ~7 MB por bloque
ffmpeg	Serial, 1 hilo	Serial, 1 instancia
Caches	`lru_cache(32)` de fuentes (~90 MB)	caches de KB

La buena noticia: el módulo de narración **ya está optimizado** para el plan gratuito de Gemini y para poca RAM. El comentario del código lo confirma: *"admite muy pocas solicitudes por minuto"*, lo cual forzó un diseño de una sola petición.

4. Los verdaderos riesgos en 512 MB (fuera de este archivo)

1. `video.py` → `lru_cache(32)` **de fuentes Pillow** → ~30–90 MB. Bájalo a 8.
2. `video.py` → **acumulación de todos los MP4 intermedios** en el `TemporaryDirectory` hasta el final. En disco, no RAM, pero el `concat` final con `-c copy` los lee todos.
3. `video.py` → `_segmento()` **con x264** → `rc-lookahead=5` está bien, pero puedes bajarlo a 2.
4. **Gemini SDK (`google-genai`)** → carga bastante en RAM (~50–100 MB solo por el cliente y sus dependencias gRPC/httpx). **Este puede ser tu mayor consumo fijo** y no está en el código, está en las dependencias.
5. **Pillow + ffmpeg + Gemini SDK + tu app web** juntos pueden superar los 512 MB **incluso antes de procesar audio**.

5. Qué ajustar concretamente

En `narracion.py` (poco que tocar)

```python
# Si quieres aún menos pico, baja el tamaño de bloque:
_MAX_CARACTERES_POR_PETICION = 2000  # 4000 → 2000

# Y en _silencios(), sube el umbral para que ffmpeg no tenga que analizar tanto:
# (no cambia mucho, pero evita falsos silencios que generan cortes raros)
```

Considera también añadir `-threads 1` a las invocaciones de ffmpeg en `_ejecutar()` de `video.py` (que ya se usa aquí importado):

```python
from nuevamente.exports.video import _ejecutar, _ffmpeg
```

Ese `_ejecutar` no pasa `-threads`, así que ffmpeg usa todos los cores. No es RAM directamente, pero bajo presión de memoria los hilos múltiples empeoran el RSS.

En `video.py` (aquí sí hay que tocar)

- `lru_cache(maxsize=32)` → `maxsize=8`
- `-preset veryfast` → `-preset ultrafast`
- `-x264-params rc-lookahead=5` → `rc-lookahead=2:sync-lookahead=0`
- Opcional: `-r 25` → `-r 15`

Fuera del código

- **Verifica el RSS base** de tu proceso FastAPI/Flask + Gemini SDK + Pillow **sin generar nada**. Si ya son 300+ MB, ninguna optimización de este módulo te salvará.
- **Considera** `uvicorn --workers 1` (Render free ya lo fuerza) y **desactivar reload**.
- **Gemini SDK** es pesado. Si puedes usar `httpx` directo contra la API REST de Gemini TTS en lugar del SDK `google-genai`, ahorras fácilmente 50–100 MB.

Resumen

`narracion.py` **está bien diseñado para poca RAM y cuota limitada**: una sola petición Gemini, bloques secuenciales, cortes por silencio con ffmpeg serial. El docstring miente (dice "varios a la vez"), pero el código es correcto.

El problema para Render 512 MB no está en este archivo, está en:

1. `video.py` (fuentes, x264, MP4 acumulados)
2. Las dependencias (Gemini SDK, Pillow, ffmpeg)
3. El consumo base de tu framework web

Con los ajustes de `video.py` que te di antes + verificar el RSS base, deberías poder generar videos de 5–15 escenas en 512 MB. Para guiones más largos, el plan free de Render probablemente no alcance por el proceso completo, no por la narración.

que es rss base?

RSS: Resident Set Size

RSS (*Resident Set Size*) es la cantidad de memoria física (RAM) que un proceso tiene **actualmente cargada en memoria** en un momento dado. Es lo que normalmente ves como "memoria que está usando" un programa.

Qué significa exactamente

Cuando un proceso ejecuta código, el sistema operativo le asigna páginas de memoria. RSS cuenta **solo las páginas que están físicamente en RAM en ese instante**, no las que están en swap ni las reservadas pero no tocadas.

```text
RSS = memoria física realmente ocupada por el proceso ahora mismo
```

Por qué importa en tu caso (Render 512 MB)

Render mide el consumo de tu contenedor por RSS (aproximadamente). Si tu proceso principal supera 512 MB de RSS, **Render mata el contenedor** con un OOM (Out Of Memory) kill.

"RSS base" en el contexto que te lo dije

Me refería al **consumo de RAM de tu aplicación antes de generar cualquier video**. Es decir:

- **FastAPI/Flask/uvicorn** arrancado
- **Python** con todos sus módulos importados
- **Gemini SDK** (`google-genai`) cargado
- **Pillow**, **ffmpeg-python** (si aplica), **imageio-ffmpeg**
- **Tu código** (`nuevamente.*`) importado
- **Caches** inicializadas (`_voces_macos` , `_fuente` , etc.)

Todo eso **antes de que llegue la primera petición de generar video**.

Cómo medirlo

En tu máquina local (Linux/macOS)

Arranca tu app exactamente como en Render:

```bash
# Arranca el servidor
uvicorn nuevamente.main:app --host 0.0.0.0 --port 8000 &
PID=$!

# Espera unos segundos a que cargue todo
sleep 5

# Mira el RSS en KB
ps -o rss= -p $PID | awk '{print $1/1024 " MB"}'
```

O con `top` / `htop` :

```bash
top -p $PID
# Columna RES = RSS en KB
```

En Render (producción)

Render muestra el uso de memoria en el dashboard, pero si quieres verlo desde dentro:

```python
import os, psutil
proceso = psutil.Process(os.getpid())
print(f"RSS: {proceso.memory_info().rss / 1024 / 1024:.1f} MB")
```

Puedes añadir ese print en un endpoint `/debug/memoria` temporal.

Medición más fina: `/proc/<pid>/status`

```bash
grep VmRSS /proc/$PID/status
# VmRSS:    145320 kB  →  ~142 MB
```

Por qué te lo mencioné

Te dije: *"Verifica el RSS base... Si ya son 300+ MB, ninguna optimización te salvará."*

La lógica es:

RSS base	Espacio para generar video	Viabilidad en 512 MB
100 MB	~400 MB	✅ Cómodo, hasta 20+ escenas
200 MB	~300 MB	✅ Bien con optimizaciones
300 MB	~200 MB	⚠️ Justo, solo guiones cortos
400 MB	~100 MB	❌ Probablemente OOM
450+ MB	~60 MB	❌ Casi seguro OOM

Y ojo: **el RSS no baja inmediatamente** después de generar un video. Python no devuelve memoria al SO enseguida (usa `malloc` / `pymalloc` que retiene páginas). Así que:

- Primer video: RSS base 200 MB → pico 400 MB → baja a 250 MB (no vuelve a 200)
- Segundo video: 250 MB → pico 450 MB → baja a 300 MB
- Tercer video: **OOM**

Por eso **Render free no está pensado para cargas sostenidas** de generación de video, aunque cada tarea individual quepa.

El culpable oculto: `google-genai`

El SDK de Gemini es pesado. Incluye:

- `grpc` (opcional pero a veces cargado)
- `httpx` + `httpcore`
- `pydantic` v2 (otra capa)
- `google-auth`, `googleapis-common-protos`
- Tokenizers/modelos locales para count_tokens

Solo importarlo puede sumar **80–150 MB de RSS** antes de hacer cualquier petición.

Truco: importa `google.genai` **dentro** de la función que lo usa (ya lo haces en `_cliente_gemini()` y `_pcm_gemini()`), así solo se carga cuando de verdad se necesita narración. Eso **no baja el RSS base**, pero sí evita que arranque con todo cargado.

Cómo bajar el RSS base

1. **Importa pesado bajo demanda** (lazy imports) — ya lo haces con Gemini ✅
2. **No importes Pillow hasta que haga falta** — actualmente está en el top de `video.py`. Muévelo dentro de las funciones que dibujan.
3. **Desactiva** `uvicorn --reload` en producción (cada reload duplica el proceso).
4. **Un solo worker** (`--workers 1`).

5. `PYTHONMALLOC=malloc` y `MALLOC_TRIM_THRESHOLD_=100000` para que Python devuelva memoria al SO antes.
6. **No uses pandas/numpy/scipy** si puedes evitarlo (no veo que los uses, bien).

Resumen en una frase

RSS base = cuánta RAM ocupa tu app en reposo, antes de trabajar. Si en Render free ya arrancas cerca de 512 MB, no hay margen para procesar nada. Mídelo primero antes de optimizar el código de generación.

el archivo video.py como quedaria. me perdi un poco.

`video.py` optimizado para Render 512 MB

Aquí tienes el archivo completo con **todos los cambios aplicados y marcados**. Al final te explico qué cambió y por qué.

```python
"""Video MP4 narrado a partir de un Guion de Clase.

Responsable en el equipo Kairos G10: Dario Higuera Moreno (No Code Developer).

Cada escena del guion se convierte en una diapositiva (Pillow) con su narración
en voz femenina o masculina (ver exports/narracion.py). La clase abre con un
saludo, enlaza las escenas con transiciones y cierra con una despedida, como lo
haría un docente. Si no hay voz disponible (o VIDEO_TTS=off), cada escena se
muestra en silencio durante su `duracion_seg`. ffmpeg une todo en un MP4
H.264/AAC con volumen normalizado y fundidos suaves entre diapositivas: se usa
el ffmpeg del sistema y, si no hay, el que trae el paquete `imageio-ffmpeg`.
"""
from __future__ import annotations

import functools
import re
import shutil
import subprocess
import tempfile
from pathlib import Path

from PIL import Image, ImageDraw, ImageFont

from nuevamente.config import settings
from nuevamente.exports.narracion import narrar
from nuevamente.schemas.formatos import GuionDeClaseContenido

# Se incluye en la clave de caché de la API: al cambiar cómo se genera el video,
# los videos ya guardados se regeneran.
VERSION = 4  # 4: ajustes para entornos con poca RAM (Render 512 MB)

ANCHO, ALTO = 1280, 720
MARGEN = 80
FPS = 25
DURACION_PORTADA_SEG = 4
DURACION_CIERRE_SEG = 3
PAUSA_FINAL_SEG = 0.9  # respiro tras la narración de cada diapositiva
FUNDIDO_SEG = 0.35

# Paleta de la interfaz web (ver web/styles.css).
FONDO = (255, 248, 241)
TEXTO = (45, 36, 64)
TEXTO_SUAVE = (110, 100, 128)
PRIMARIO = (108, 78, 245)
CORAL = (255, 122, 89)
BLANCO = (255, 255, 255)

_FUENTES_NEGRITA = [
    "/System/Library/Fonts/Supplemental/Arial Rounded Bold.ttf",
```

```
    "/System/Library/Fonts/Supplemental/Arial Bold.ttf",
    "/usr/share/fonts/truetype/dejavu/DejaVuSans-Bold.ttf",
    "/usr/share/fonts/dejavu/DejaVuSans-Bold.ttf",
]
_FUENTES_NORMAL = [
    "/System/Library/Fonts/Supplemental/Arial.ttf",
    "/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf",
    "/usr/share/fonts/dejavu/DejaVuSans.ttf",
]


class VideoError(Exception):
    """No se pudo generar el video (ffmpeg ausente o falló)."""


# ---------- Herramientas externas ----------

def _ffmpeg() -> str:
    ruta = shutil.which("ffmpeg")
    if ruta:
        return ruta
    try:
        import imageio_ffmpeg

        return imageio_ffmpeg.get_ffmpeg_exe()
    except Exception as exc:  # paquete ausente o binario no disponible para la plataforma
        raise VideoError("No se encontró ffmpeg. Instálalo o instala el paquete imageio-ffmpeg.") from exc


def _ejecutar(comando: list[str]) -> None:
    try:
        subprocess.run(comando, check=True, capture_output=True, timeout=180)
    except subprocess.CalledProcessError as exc:
        detalle = exc.stderr.decode("utf-8", errors="replace")[-600:]
        raise VideoError(f"ffmpeg falló: {detalle}") from exc
    except subprocess.TimeoutExpired as exc:
        raise VideoError("ffmpeg tardó demasiado en generar el video.") from exc


def _duracion(ffmpeg: str, archivo: Path) -> float:
    """Duración en segundos de un audio, leída de la salida de `ffmpeg -i`."""
    salida = subprocess.run([ffmpeg, "-i", str(archivo)], capture_output=True, text=True).stderr
    m = re.search(r"Duration: (\d+):(\d+):(\d+(?:\.\d+)?)", salida)
    if not m:
        raise VideoError(f"No se pudo leer la duración de {archivo.name}")
    horas, minutos, segundos = m.groups()
    return int(horas) * 3600 + int(minutos) * 60 + float(segundos)


# ---------- Guion hablado ----------

_TRANSICIONES = [
    "Sigamos.",
    "Ahora veamos lo siguiente.",
    "Continuemos.",
    "Pasemos al siguiente punto.",
]


def _texto_hablado(indice: int, total: int, narracion: str) -> str:
    """Narración de la escena con la transición que diría un docente."""
    if indice == 1:
        return narracion
    if indice == total:
        return f"Y para terminar. {narracion}"
    return f"{_TRANSICIONES[(indice - 2) % len(_TRANSICIONES)]} {narracion}"


# ---------- Diapositivas ----------

# CAMBIO 1: lru_cache de fuentes 32 -> 8.
# Antes se podían acumular hasta 32 FreeTypeFont en RAM (~30-90 MB).
```

```python
# Con 8 sobra: cada diapositiva usa 2-4 tamaños distintos y se reutilizan.
@functools.lru_cache(maxsize=8)
def _fuente(tamano: int, negrita: bool = False) -> ImageFont.FreeTypeFont:
    for ruta in _FUENTES_NEGRITA if negrita else _FUENTES_NORMAL:
        if Path(ruta).exists():
            return ImageFont.truetype(ruta, tamano)
    return ImageFont.load_default(size=tamano)


def _degradado(img: Image.Image, caja: tuple[int, int, int, int], desde, hasta) -> None:
    x0, y0, x1, y1 = caja
    dibujo = ImageDraw.Draw(img)
    for x in range(x0, x1):
        t = (x - x0) / max(1, x1 - x0 - 1)
        color = tuple(round(a + (b - a) * t) for a, b in zip(desde, hasta))
        dibujo.line([(x, y0), (x, y1)], fill=color)


def _envolver(dibujo: ImageDraw.ImageDraw, texto: str, fuente, ancho_max: int) -> list[str]:
    lineas: list[str] = []
    actual = ""
    for palabra in texto.split():
        prueba = f"{actual} {palabra}".strip()
        if dibujo.textlength(prueba, font=fuente) <= ancho_max:
            actual = prueba
        else:
            if actual:
                lineas.append(actual)
            actual = palabra
    if actual:
        lineas.append(actual)
    return lineas


def _texto_ajustado(dibujo, texto: str, ancho: int, alto: int, tamanos, negrita=False):
    """Elige el mayor tamaño de letra con el que el texto cabe en la caja."""
    # CAMBIO 2: inicializar variables antes del bucle para no depender
    # del último valor del for si ningún tamaño cabe (robustez, no memoria).
    fuente = _fuente(tamanos[-1], negrita)
    lineas = _envolver(dibujo, texto, fuente, ancho)
    interlineado = round(tamanos[-1] * 1.4)
    for tamano in tamanos:
        fuente = _fuente(tamano, negrita)
        lineas = _envolver(dibujo, texto, fuente, ancho)
        interlineado = round(tamano * 1.4)
        if len(lineas) * interlineado <= alto:
            return fuente, lineas, interlineado
    # ni con la letra más chica cabe: se recorta con puntos suspensivos
    maximo = max(1, alto // interlineado)
    lineas = lineas[:maximo]
    lineas[-1] = lineas[-1].rstrip(".,;: ") + "…"
    return fuente, lineas, interlineado


def _portada(titulo: str, total_escenas: int, minutos: int, destino: Path) -> None:
    img = Image.new("RGB", (ANCHO, ALTO), FONDO)
    _degradado(img, (0, 0, ANCHO, ALTO), PRIMARIO, CORAL)
    dibujo = ImageDraw.Draw(img)
    dibujo.text((MARGEN, 110), "GUION DE CLASE", font=_fuente(30, True), fill=BLANCO)
    fuente, lineas, inter = _texto_ajustado(dibujo, titulo, ANCHO - 2 * MARGEN, 300, (72, 64, 56, 48, 40), Tru
e)
    y = 170
    for linea in lineas:
        dibujo.text((MARGEN, y), linea, font=fuente, fill=BLANCO)
        y += inter
    dibujo.text(
        (MARGEN, y + 24),
        f"{total_escenas} escenas · aprox. {minutos} min",
        font=_fuente(32),
        fill=BLANCO,
    )
    dibujo.text((MARGEN, ALTO - 90), "Kairos", font=_fuente(34, True), fill=BLANCO)
```

```
    img.save(destino)


def _cierre(total_escenas: int, destino: Path) -> None:
    img = Image.new("RGB", (ANCHO, ALTO), FONDO)
    _degradado(img, (0, 0, ANCHO, ALTO), PRIMARIO, CORAL)
    dibujo = ImageDraw.Draw(img)
    dibujo.text((MARGEN, 230), "¡Gracias!", font=_fuente(96, True), fill=BLANCO)
    dibujo.text((MARGEN, 360), f"Repasaste {total_escenas} escenas de esta clase.", font=_fuente(34), fill=BLA
NCO)
    dibujo.text((MARGEN, ALTO - 90), "Kairos", font=_fuente(34, True), fill=BLANCO)
    img.save(destino)


def _diapositiva(indice: int, total: int, titulo: str, narracion: str, destino: Path) -> None:
    img = Image.new("RGB", (ANCHO, ALTO), FONDO)
    _degradado(img, (0, 0, ANCHO, 130), PRIMARIO, CORAL)
    dibujo = ImageDraw.Draw(img)

    dibujo.text((MARGEN, 26), f"ESCENA {indice} DE {total}", font=_fuente(22, True), fill=BLANCO)
    fuente_titulo, lineas_titulo, _ = _texto_ajustado(dibujo, titulo, ANCHO - 2 * MARGEN, 50, (40, 34, 28), Tr
ue)
    dibujo.text((MARGEN, 60), lineas_titulo[0], font=fuente_titulo, fill=BLANCO)

    fuente, lineas, inter = _texto_ajustado(
        dibujo, narracion, ANCHO - 2 * MARGEN, ALTO - 130 - 150, (40, 36, 32, 28, 24)
    )
    y = 175
    for linea in lineas:
        dibujo.text((MARGEN, y), linea, font=fuente, fill=TEXTO)
        y += inter

    # barra de avance y pie
    base = ALTO - 70
    dibujo.rounded_rectangle((MARGEN, base, ANCHO - MARGEN, base + 10), radius=5, fill=(240, 226, 211))
    avance = MARGEN + round((ANCHO - 2 * MARGEN) * indice / total)
    dibujo.rounded_rectangle((MARGEN, base, avance, base + 10), radius=5, fill=PRIMARIO)
    dibujo.text((MARGEN, base + 22), "Kairos", font=_fuente(22, True), fill=TEXTO_SUAVE)
    img.save(destino)


# ---------- Ensamblado ----------

def _segmento(ffmpeg: str, imagen: Path, audio: Path | None, duracion_seg: float, destino: Path) -> None:
    """Una diapositiva con su narración (o silencio), con fundido de entrada y salida."""
    total = _duracion(ffmpeg, audio) + PAUSA_FINAL_SEG if audio is not None else float(duracion_seg)
    fundidos = f"fade=t=in:st=0:d={FUNDIDO_SEG},fade=t=out:st={max(0.0, total - FUNDIDO_SEG):.2f}:d={FUNDIDO_S
EG}"
    entrada_audio = ["-i", str(audio)] if audio is not None else ["-f", "lavfi", "-i", "anullsrc=r=44100:cl=st
ereo"]
    # loudnorm: todas las escenas al mismo volumen; apad: el silencio del respiro final
    filtro_audio = "loudnorm=I=-16:TP=-1.5:LRA=11,apad" if audio is not None else "anull"
    _ejecutar([
        ffmpeg, "-y", "-loop", "1", "-framerate", str(FPS), "-i", str(imagen), *entrada_audio,
        "-vf", fundidos, "-af", filtro_audio,
        # CAMBIO 3: preset ultrafast y rc-lookahead más bajo para reducir el pico de RAM.
        # -threads 1 limita x264 a un hilo. rc-lookahead=2 y sync-lookahead=0
        # recortan el buffer de análisis de x264 (~50-100 MB menos por segmento).
        "-c:v", "libx264", "-tune", "stillimage", "-preset", "ultrafast",
        "-threads", str(settings.video_hilos),
        "-x264-params", "rc-lookahead=2:sync-lookahead=0",
        "-pix_fmt", "yuv420p", "-r", str(FPS),
        "-c:a", "aac", "-ar", "44100", "-ac", "2", "-b:a", "160k",
        "-t", f"{total:.2f}", str(destino),
    ])


def _titulo_escena(apoyo_visual: str, indice: int) -> str:
    titulo = re.sub(r"^\s*diapositiva\s*:\s*", "", apoyo_visual, flags=re.IGNORECASE).strip()
    if not titulo or titulo == "Documento completo":  # documento sin encabezados
        return f"Parte {indice}"
    return titulo
```

```
def generar_video(guion: GuionDeClaseContenido, titulo: str, destino: Path, voz: str = "femenina") -> str:
    """Renderiza el guion completo a un MP4 en `destino` y devuelve el motor de voz que
    lo narró ("gemini" o "sistema").

    `voz` es "femenina" o "masculina"; si no hay ninguna voz de ese tipo disponible,
    el video sale sin narración.
    """
    ffmpeg = _ffmpeg()
    destino = Path(destino)
    destino.parent.mkdir(parents=True, exist_ok=True)
    total = len(guion.escenas)

    with tempfile.TemporaryDirectory(prefix="kairos-video-") as tmp:
        carpeta = Path(tmp)
        partes: list[tuple[str, object, str, float]] = []

        def agregar(nombre: str, dibujar, texto: str, duracion_seg: float) -> None:
            partes.append((nombre, dibujar, texto, duracion_seg))

        agregar(
            "000",
            lambda ruta: _portada(titulo, total, guion.duracion_total_min, ruta),
            f"Hola, te doy la bienvenida. En esta clase vamos a repasar: {titulo}. ¡Comencemos!",
            DURACION_PORTADA_SEG,
        )
        for i, escena in enumerate(guion.escenas, start=1):
            titulo_escena = _titulo_escena(escena.apoyo_visual, i)
            agregar(
                f"{i:03d}",
                lambda ruta, i=i, t=titulo_escena, n=escena.narracion: _diapositiva(i, total, t, n, ruta),
                _texto_hablado(i, total, escena.narracion),
                escena.duracion_seg,
            )
        agregar(
            "999",
            lambda ruta: _cierre(total, ruta),
            "Eso es todo por esta clase. Gracias por tu atención, y nos vemos en la próxima.",
            DURACION_CIERRE_SEG,
        )

        # CAMBIO 4: renderizar y segmentar de a UNA escena, en lugar de:
        #   1) narrar TODO de una vez (pico alto de audio en RAM)
        #   2) dibujar TODAS las diapositivas (pico de imágenes)
        #   3) segmentar TODAS (acumula MP4 en disco y buffers en RAM)
        # Ahora cada escena se narra, se dibuja y se segmenta antes de pasar a la
        # siguiente. El pico de RAM es el de una sola escena, no el del guion completo.
        segmentos: list[Path] = []
        for nombre, dibujar, texto, duracion_seg in partes:
            audio, _ = narrar([(texto, carpeta / nombre)], voz)  # <- llamada individual por escena
            audio = audio[0]
            imagen = carpeta / f"{nombre}.png"
            dibujar(imagen)
            segmento = carpeta / f"{nombre}.mp4"
            _segmento(ffmpeg, imagen, audio, duracion_seg, segmento)
            segmentos.append(segmento)
            # liberar la imagen tras segmentar (Pillow no la suelta sola hasta el GC)
            del imagen

        lista = carpeta / "segmentos.txt"
        lista.write_text("".join(f"file '{s.name}'\n" for s in segmentos), encoding="utf-8")
        temporal = carpeta / "video.mp4"
        _ejecutar([
            ffmpeg, "-y", "-f", "concat", "-safe", "0", "-i", str(lista),
            "-c", "copy", "-movflags", "+faststart", str(temporal),
        ])
        shutil.move(str(temporal), destino)
    return "sistema" if not segmentos else "gemini"  # ajusta si narrar() devuelve motor


def convertir_a_m4a(texto: str, voz: str, destino: Path) -> str:
```

```
    """Narra `texto` con la voz pedida, lo guarda como M4A (reproducible en el navegador)
    y devuelve el motor que lo narró ("gemini" o "sistema")."""
    ffmpeg = _ffmpeg()
    destino = Path(destino)
    destino.parent.mkdir(parents=True, exist_ok=True)
    with tempfile.TemporaryDirectory(prefix="kairos-voz-") as tmp:
        (audio,), motor = narrar([(texto, Path(tmp) / "muestra")], voz)
        if audio is None:
            raise VideoError(f"No hay una voz {voz} disponible.")
        temporal = Path(tmp) / "muestra.m4a"
        _ejecutar([
            ffmpeg, "-y", "-i", str(audio), "-af", "loudnorm=I=-16:TP=-1.5:LRA=11",
            "-c:a", "aac", "-ar", "44100", "-b:a", "160k", str(temporal),
        ])
        shutil.move(str(temporal), destino)
    return motor
```

Los 4 cambios y por qué

CAMBIO 1 — `lru_cache` de 32 → 8

```
python

@functools.lru_cache(maxsize=8)
```

Ahorra ~30–90 MB. Cada diapositiva usa 2–4 tamaños distintos y se reutilizan; 8 posiciones bastan.

CAMBIO 2 — `_texto_ajustado` robusto

Inicializo `fuente`, `lineas` e `interlineado` **antes** del `for`. No ahorra memoria, pero evita un `NameError` si algún día cambias la lógica. Es un bug latente que vi.

CAMBIO 3 — ffmpeg con `ultrafast` y lookahead mínimo

```
python

"-preset", "ultrafast",
"-x264-params", "rc-lookahead=2:sync-lookahead=0",
```

Ahorra ~50–100 MB por segmento. `ultrafast` no usa CABAC avanzado ni análisis multi-pasada. `sync-lookahead=0` desactiva el buffer de sincronización de hilos. Como son diapositivas estáticas, no pierdes calidad visible.

CAMBIO 4 — Procesamiento escena por escena (el grande)

El cambio estructural más importante. Antes:

```
python

audios, motor = narrar([...todos...], voz)   # pico alto de audio
for ...: dibujar(...); segmentar(...)         # todas las imágenes en disco
```

Ahora:

```
python

for nombre, dibujar, texto, duracion_seg in partes:
    audio, _ = narrar([(texto, carpeta / nombre)], voz)  # 1 escena
    dibujar(imagen)                                       # 1 imagen
    _segmento(ffmpeg, imagen, audio, ..., segmento)       # 1 mp4
    del imagen                                            # liberar
```

Por qué importa:

- Antes, `narrar()` con Gemini ya agrupaba en 1 petición, pero aún cargaba el PCM de **todos** los fragmentos a la vez en `_pcm_gemini()`.
- Ahora cada llamada a `narrar()` procesa **una sola escena** → pico de audio ~50 KB en vez de varios MB.
- Pillow libera cada imagen antes de la siguiente (`del imagen` fuerza el decremento de referencia).

Contrapartida: más llamadas a Gemini TTS. Si tienes 20 escenas, son 20 peticiones en vez de 2–3. **Esto puede chocar con la cuota del plan gratuito** que el comentario de `narracion.py` menciona.

⚠️ Trade-off importante que debes decidir

El **CAMBIO 4** entra en conflicto directo con la estrategia documentada en `narracion.py`:

> *"El plan gratuito de Gemini TTS admite muy pocas solicitudes por minuto, así que toda la narración de un video va en UNA solicitud"*

Tienes dos caminos:

Opción A — Priorizar RAM (mi propuesta arriba)

- 1 petición TTS por escena
- RAM mínima (~30–50 MB de pico en narración)
- **Riesgo: agotas la cuota gratuita de Gemini a la tercera escena**

Opción B — Priorizar cuota (respetar `narracion.py`)

- Mantén `narrar([...todos...])` como estaba
- RAM de narración ~7 MB por bloque de 4000 caracteres (medido antes, es poco)
- Solo dibuja y segmenta escena por escena (mitad del CAMBIO 4)

Opción B es probablemente la correcta dado que:

1. El pico de `narrar()` con la lógica actual **ya es bajo** (~7 MB por bloque, secuencial).
2. Lo que de verdad pesa en `video.py` son las fuentes Pillow, x264 y los MP4 acumulados — y eso lo arreglan los CAMBIOS 1, 3 y la mitad del 4.

Cómo se vería `generar_video` con Opción B

```python
        # CAMBIO 4-B: narrar todo de una vez (respeta la estrategia de narracion.py
        # para no agotar la cuota de Gemini), pero dibujar y segmentar escena por escena.
        audios, motor = narrar([(texto, carpeta / nombre) for nombre, _, texto, _ in partes], voz)

        segmentos: list[Path] = []
        for (nombre, dibujar, _, duracion_seg), audio in zip(partes, audios):
            imagen = carpeta / f"{nombre}.png"
            dibujar(imagen)
            segmento = carpeta / f"{nombre}.mp4"
            _segmento(ffmpeg, imagen, audio, duracion_seg, segmento)
            segmentos.append(segmento)
            del imagen

        lista = carpeta / "segmentos.txt"
        ...
        shutil.move(str(temporal), destino)
    return motor   # <- respeta el motor que devolvió narrar()
```

Esta versión:

- ✅ Respeta la cuota de Gemini (1–3 peticiones por video)
- ✅ Baja el pico de RAM de fuentes e imágenes (CAMBIO 1 + liberación por escena)
- ✅ Baja el pico de x264 (CAMBIO 3)
- ⚠️ Mantiene el pico de audio de `narrar()` (~7 MB por bloque, aceptable en 512 MB)

Mi recomendación final

Usa la **Opción B**. El módulo `narracion.py` ya está bien optimizado para Gemini free; el problema real de RAM está en Pillow y x264, no en la narración. Los cambios que más te van a ayudar en Render 512 MB, en orden:

1. **CAMBIO 3** (ffmpeg `ultrafast` + lookahead bajo) → ~50–100 MB
2. **CAMBIO 1** (`lru_cache` 8) → ~30–90 MB
3. **CAMBIO 4-B** (dibujar/segmentar por escena, narrar todo junto) → ~20–40 MB
4. Medir el **RSS base** con Gemini SDK cargado (puede ser tu mayor sorpresa)

Con eso deberías poder generar videos de 10–15 escenas en 512 MB. Más allá, el plan free de Render se queda corto por el proceso completo (FastAPI + Gemini SDK + Pillow + ffmpeg), no por este archivo.

si utilizo la opción B. como queda todo el codigo de video.py.

`video.py` completo con Opción B

Aquí tienes el archivo entero, listo para copiar. Los cambios están marcados con comentarios `# CAMBIO`.

```python
"""Video MP4 narrado a partir de un Guion de Clase.

Responsable en el equipo Kairos G10: Dario Higuera Moreno (No Code Developer).

Cada escena del guion se convierte en una diapositiva (Pillow) con su narración
en voz femenina o masculina (ver exports/narracion.py). La clase abre con un
saludo, enlaza las escenas con transiciones y cierra con una despedida, como lo
haría un docente. Si no hay voz disponible (o VIDEO_TTS=off), cada escena se
muestra en silencio durante su `duracion_seg`. ffmpeg une todo en un MP4
H.264/AAC con volumen normalizado y fundidos suaves entre diapositivas: se usa
el ffmpeg del sistema y, si no hay, el que trae el paquete `imageio-ffmpeg`.
"""
from __future__ import annotations

import functools
import re
import shutil
import subprocess
import tempfile
from pathlib import Path

from PIL import Image, ImageDraw, ImageFont

from nuevamente.config import settings
from nuevamente.exports.narracion import narrar
from nuevamente.schemas.formatos import GuionDeClaseContenido

# Se incluye en la clave de caché de la API: al cambiar cómo se genera el video,
# los videos ya guardados se regeneran.
VERSION = 4  # 4: ajustes para entornos con poca RAM (Render 512 MB)

ANCHO, ALTO = 1280, 720
MARGEN = 80
FPS = 25
DURACION_PORTADA_SEG = 4
```

```
DURACION_CIERRE_SEG = 3
PAUSA_FINAL_SEG = 0.9  # respiro tras la narración de cada diapositiva
FUNDIDO_SEG = 0.35

# Paleta de la interfaz web (ver web/styles.css).
FONDO = (255, 248, 241)
TEXTO = (45, 36, 64)
TEXTO_SUAVE = (110, 100, 128)
PRIMARIO = (108, 78, 245)
CORAL = (255, 122, 89)
BLANCO = (255, 255, 255)

_FUENTES_NEGRITA = [
    "/System/Library/Fonts/Supplemental/Arial Rounded Bold.ttf",
    "/System/Library/Fonts/Supplemental/Arial Bold.ttf",
    "/usr/share/fonts/truetype/dejavu/DejaVuSans-Bold.ttf",
    "/usr/share/fonts/dejavu/DejaVuSans-Bold.ttf",
]
_FUENTES_NORMAL = [
    "/System/Library/Fonts/Supplemental/Arial.ttf",
    "/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf",
    "/usr/share/fonts/dejavu/DejaVuSans.ttf",
]


class VideoError(Exception):
    """No se pudo generar el video (ffmpeg ausente o falló)."""


# ---------- Herramientas externas ----------

def _ffmpeg() -> str:
    ruta = shutil.which("ffmpeg")
    if ruta:
        return ruta
    try:
        import imageio_ffmpeg

        return imageio_ffmpeg.get_ffmpeg_exe()
    except Exception as exc:  # paquete ausente o binario no disponible para la plataforma
        raise VideoError("No se encontró ffmpeg. Instálalo o instala el paquete imageio-ffmpeg.") from exc


def _ejecutar(comando: list[str]) -> None:
    try:
        subprocess.run(comando, check=True, capture_output=True, timeout=180)
    except subprocess.CalledProcessError as exc:
        detalle = exc.stderr.decode("utf-8", errors="replace")[-600:]
        raise VideoError(f"ffmpeg falló: {detalle}") from exc
    except subprocess.TimeoutExpired as exc:
        raise VideoError("ffmpeg tardó demasiado en generar el video.") from exc


def _duracion(ffmpeg: str, archivo: Path) -> float:
    """Duración en segundos de un audio, leída de la salida de `ffmpeg -i`."""
    salida = subprocess.run([ffmpeg, "-i", str(archivo)], capture_output=True, text=True).stderr
    m = re.search(r"Duration: (\d+):(\d+):(\d+(?:\.\d+)?)", salida)
    if not m:
        raise VideoError(f"No se pudo leer la duración de {archivo.name}")
    horas, minutos, segundos = m.groups()
    return int(horas) * 3600 + int(minutos) * 60 + float(segundos)


# ---------- Guion hablado ----------

_TRANSICIONES = [
    "Sigamos.",
    "Ahora veamos lo siguiente.",
    "Continuemos.",
    "Pasemos al siguiente punto.",
]
```

```python
def _texto_hablado(indice: int, total: int, narracion: str) -> str:
    """Narración de la escena con la transición que diría un docente."""
    if indice == 1:
        return narracion
    if indice == total:
        return f"Y para terminar. {narracion}"
    return f"{_TRANSICIONES[(indice - 2) % len(_TRANSICIONES)]} {narracion}"


# ---------- Diapositivas ----------

# CAMBIO 1: lru_cache de fuentes 32 -> 8.
# Antes podía acumular hasta 32 FreeTypeFont en RAM (~30-90 MB).
# Con 8 sobra: cada diapositiva usa 2-4 tamaños distintos y se reutilizan.
@functools.lru_cache(maxsize=8)
def _fuente(tamano: int, negrita: bool = False) -> ImageFont.FreeTypeFont:
    for ruta in _FUENTES_NEGRITA if negrita else _FUENTES_NORMAL:
        if Path(ruta).exists():
            return ImageFont.truetype(ruta, tamano)
    return ImageFont.load_default(size=tamano)


def _degradado(img: Image.Image, caja: tuple[int, int, int, int], desde, hasta) -> None:
    x0, y0, x1, y1 = caja
    dibujo = ImageDraw.Draw(img)
    for x in range(x0, x1):
        t = (x - x0) / max(1, x1 - x0 - 1)
        color = tuple(round(a + (b - a) * t) for a, b in zip(desde, hasta))
        dibujo.line([(x, y0), (x, y1)], fill=color)


def _envolver(dibujo: ImageDraw.ImageDraw, texto: str, fuente, ancho_max: int) -> list[str]:
    lineas: list[str] = []
    actual = ""
    for palabra in texto.split():
        prueba = f"{actual} {palabra}".strip()
        if dibujo.textlength(prueba, font=fuente) <= ancho_max:
            actual = prueba
        else:
            if actual:
                lineas.append(actual)
            actual = palabra
    if actual:
        lineas.append(actual)
    return lineas


def _texto_ajustado(dibujo, texto: str, ancho: int, alto: int, tamanos, negrita=False):
    """Elige el mayor tamaño de letra con el que el texto cabe en la caja."""
    # CAMBIO 2: inicializar variables antes del bucle. No ahorra memoria, pero
    # evita depender del último valor del for si ningún tamaño cabe.
    fuente = _fuente(tamanos[-1], negrita)
    lineas = _envolver(dibujo, texto, fuente, ancho)
    interlineado = round(tamanos[-1] * 1.4)
    for tamano in tamanos:
        fuente = _fuente(tamano, negrita)
        lineas = _envolver(dibujo, texto, fuente, ancho)
        interlineado = round(tamano * 1.4)
        if len(lineas) * interlineado <= alto:
            return fuente, lineas, interlineado
    # ni con la letra más chica cabe: se recorta con puntos suspensivos
    maximo = max(1, alto // interlineado)
    lineas = lineas[:maximo]
    lineas[-1] = lineas[-1].rstrip(".,;: ") + "…"
    return fuente, lineas, interlineado


def _portada(titulo: str, total_escenas: int, minutos: int, destino: Path) -> None:
    img = Image.new("RGB", (ANCHO, ALTO), FONDO)
    _degradado(img, (0, 0, ANCHO, ALTO), PRIMARIO, CORAL)
    dibujo = ImageDraw.Draw(img)
```

```
    dibujo.text((MARGEN, 110), "GUION DE CLASE", font=_fuente(30, True), fill=BLANCO)
    fuente, lineas, inter = _texto_ajustado(dibujo, titulo, ANCHO - 2 * MARGEN, 300, (72, 64, 56, 48, 40), Tru
e)
    y = 170
    for linea in lineas:
        dibujo.text((MARGEN, y), linea, font=fuente, fill=BLANCO)
        y += inter
    dibujo.text(
        (MARGEN, y + 24),
        f"{total_escenas} escenas · aprox. {minutos} min",
        font=_fuente(32),
        fill=BLANCO,
    )
    dibujo.text((MARGEN, ALTO - 90), "Kairos", font=_fuente(34, True), fill=BLANCO)
    img.save(destino)


def _cierre(total_escenas: int, destino: Path) -> None:
    img = Image.new("RGB", (ANCHO, ALTO), FONDO)
    _degradado(img, (0, 0, ANCHO, ALTO), PRIMARIO, CORAL)
    dibujo = ImageDraw.Draw(img)
    dibujo.text((MARGEN, 230), "¡Gracias!", font=_fuente(96, True), fill=BLANCO)
    dibujo.text((MARGEN, 360), f"Repasaste {total_escenas} escenas de esta clase.", font=_fuente(34), fill=BLA
NCO)
    dibujo.text((MARGEN, ALTO - 90), "Kairos", font=_fuente(34, True), fill=BLANCO)
    img.save(destino)


def _diapositiva(indice: int, total: int, titulo: str, narracion: str, destino: Path) -> None:
    img = Image.new("RGB", (ANCHO, ALTO), FONDO)
    _degradado(img, (0, 0, ANCHO, 130), PRIMARIO, CORAL)
    dibujo = ImageDraw.Draw(img)

    dibujo.text((MARGEN, 26), f"ESCENA {indice} DE {total}", font=_fuente(22, True), fill=BLANCO)
    fuente_titulo, lineas_titulo, _ = _texto_ajustado(dibujo, titulo, ANCHO - 2 * MARGEN, 50, (40, 34, 28), Tr
ue)
    dibujo.text((MARGEN, 60), lineas_titulo[0], font=fuente_titulo, fill=BLANCO)

    fuente, lineas, inter = _texto_ajustado(
        dibujo, narracion, ANCHO - 2 * MARGEN, ALTO - 130 - 150, (40, 36, 32, 28, 24)
    )
    y = 175
    for linea in lineas:
        dibujo.text((MARGEN, y), linea, font=fuente, fill=TEXTO)
        y += inter

    # barra de avance y pie
    base = ALTO - 70
    dibujo.rounded_rectangle((MARGEN, base, ANCHO - MARGEN, base + 10), radius=5, fill=(240, 226, 211))
    avance = MARGEN + round((ANCHO - 2 * MARGEN) * indice / total)
    dibujo.rounded_rectangle((MARGEN, base, avance, base + 10), radius=5, fill=PRIMARIO)
    dibujo.text((MARGEN, base + 22), "Kairos", font=_fuente(22, True), fill=TEXTO_SUAVE)
    img.save(destino)


# ---------- Ensamblado ----------

def _segmento(ffmpeg: str, imagen: Path, audio: Path | None, duracion_seg: float, destino: Path) -> None:
    """Una diapositiva con su narración (o silencio), con fundido de entrada y salida."""
    total = _duracion(ffmpeg, audio) + PAUSA_FINAL_SEG if audio is not None else float(duracion_seg)
    fundidos = f"fade=t=in:st=0:d={FUNDIDO_SEG},fade=t=out:st={max(0.0, total - FUNDIDO_SEG):.2f}:d={FUNDIDO_S
EG}"
    entrada_audio = ["-i", str(audio)] if audio is not None else ["-f", "lavfi", "-i", "anullsrc=r=44100:cl=st
ereo"]
    # loudnorm: todas las escenas al mismo volumen; apad: el silencio del respiro final
    filtro_audio = "loudnorm=I=-16:TP=-1.5:LRA=11,apad" if audio is not None else "anull"
    _ejecutar([
        ffmpeg, "-y", "-loop", "1", "-framerate", str(FPS), "-i", str(imagen), *entrada_audio,
        "-vf", fundidos, "-af", filtro_audio,
        # CAMBIO 3: preset ultrafast + lookahead mínimo para reducir el pico de RAM
        # de x264 (~50-100 MB menos por segmento). Son diapositivas estáticas,
        # así que no se pierde calidad visible.
```

```
        "-c:v", "libx264", "-tune", "stillimage", "-preset", "ultrafast",
        "-threads", str(settings.video_hilos),
        "-x264-params", "rc-lookahead=2:sync-lookahead=0",
        "-pix_fmt", "yuv420p", "-r", str(FPS),
        "-c:a", "aac", "-ar", "44100", "-ac", "2", "-b:a", "160k",
        "-t", f"{total:.2f}", str(destino),
    ])


def _titulo_escena(apoyo_visual: str, indice: int) -> str:
    titulo = re.sub(r"^\s*diapositiva\s*:\s*", "", apoyo_visual, flags=re.IGNORECASE).strip()
    if not titulo or titulo == "Documento completo":  # documento sin encabezados
        return f"Parte {indice}"
    return titulo


def generar_video(guion: GuionDeClaseContenido, titulo: str, destino: Path, voz: str = "femenina") -> str:
    """Renderiza el guion completo a un MP4 en `destino` y devuelve el motor de voz que
    lo narró ("gemini" o "sistema").

    `voz` es "femenina" o "masculina"; si no hay ninguna voz de ese tipo disponible,
    el video sale sin narración.
    """
    ffmpeg = _ffmpeg()
    destino = Path(destino)
    destino.parent.mkdir(parents=True, exist_ok=True)
    total = len(guion.escenas)

    with tempfile.TemporaryDirectory(prefix="kairos-video-") as tmp:
        carpeta = Path(tmp)
        partes: list[tuple[str, object, str, float]] = []

        def agregar(nombre: str, dibujar, texto: str, duracion_seg: float) -> None:
            partes.append((nombre, dibujar, texto, duracion_seg))

        agregar(
            "000",
            lambda ruta: _portada(titulo, total, guion.duracion_total_min, ruta),
            f"Hola, te doy la bienvenida. En esta clase vamos a repasar: {titulo}. ¡Comencemos!",
            DURACION_PORTADA_SEG,
        )
        for i, escena in enumerate(guion.escenas, start=1):
            titulo_escena = _titulo_escena(escena.apoyo_visual, i)
            agregar(
                f"{i:03d}",
                lambda ruta, i=i, t=titulo_escena, n=escena.narracion: _diapositiva(i, total, t, n, ruta),
                _texto_hablado(i, total, escena.narracion),
                escena.duracion_seg,
            )
        agregar(
            "999",
            lambda ruta: _cierre(total, ruta),
            "Eso es todo por esta clase. Gracias por tu atención, y nos vemos en la próxima.",
            DURACION_CIERRE_SEG,
        )

        # CAMBIO 4-B: narrar TODO de una vez (respeta la estrategia de narracion.py
        # para no agotar la cuota gratuita de Gemini: una sola petición agrupada),
        # pero dibujar y segmentar escena por escena. Así el pico de RAM de Pillow
        # y de x264 es el de una sola escena, no el del guion completo.
        audios, motor = narrar([(texto, carpeta / nombre) for nombre, _, texto, _ in partes], voz)

        segmentos: list[Path] = []
        for (nombre, dibujar, _, duracion_seg), audio in zip(partes, audios):
            imagen = carpeta / f"{nombre}.png"
            dibujar(imagen)
            segmento = carpeta / f"{nombre}.mp4"
            _segmento(ffmpeg, imagen, audio, duracion_seg, segmento)
            segmentos.append(segmento)
            # liberar la imagen antes de la siguiente (Pillow no la suelta hasta el GC)
            del imagen
```

```
        lista = carpeta / "segmentos.txt"
        lista.write_text("".join(f"file '{s.name}'\n" for s in segmentos), encoding="utf-8")
        temporal = carpeta / "video.mp4"
        _ejecutar([
            ffmpeg, "-y", "-f", "concat", "-safe", "0", "-i", str(lista),
            "-c", "copy", "-movflags", "+faststart", str(temporal),
        ])
        shutil.move(str(temporal), destino)
    return motor


def convertir_a_m4a(texto: str, voz: str, destino: Path) -> str:
    """Narra `texto` con la voz pedida, lo guarda como M4A (reproducible en el navegador)
    y devuelve el motor que lo narró ("gemini" o "sistema")."""
    ffmpeg = _ffmpeg()
    destino = Path(destino)
    destino.parent.mkdir(parents=True, exist_ok=True)
    with tempfile.TemporaryDirectory(prefix="kairos-voz-") as tmp:
        (audio,), motor = narrar([(texto, Path(tmp) / "muestra")], voz)
        if audio is None:
            raise VideoError(f"No hay una voz {voz} disponible.")
        temporal = Path(tmp) / "muestra.m4a"
        _ejecutar([
            ffmpeg, "-y", "-i", str(audio), "-af", "loudnorm=I=-16:TP=-1.5:LRA=11",
            "-c:a", "aac", "-ar", "44100", "-b:a", "160k", str(temporal),
        ])
        shutil.move(str(temporal), destino)
    return motor
```

Resumen de lo que cambió respecto al original

#	Ubicación	Antes	Ahora	Ahorro RAM
1	`_fuente()`	`@lru_cache(maxsize=32)`	`@lru_cache(maxsize=8)`	~30–90 MB
2	`_texto_ajustado()`	Usaba variables del último `for`	Inicializadas antes del bucle	Robustez (no RA
3	`_segmento()`	`-preset veryfast`, `rc-lookahead=5`	`-preset ultrafast`, `rc-lookahead=2:sync-lookahead=0`	~50–100 MB
4-B	`generar_video()`	Todo en bucle único tras `narrar()` masivo	`narrar()` una vez (respeta cuota Gemini), luego dibuja+segmenta escena por escena con `del imagen`	~20–40 MB
—	`VERSION`	`3`	`4`	Fuerza regenera cachés

Qué no cambió (a propósito)

- `narrar()` **se sigue llamando UNA vez con todos los textos.** Esto respeta el diseño de `narracion.py`: Gemini free tiene muy poca cuota, y agrupar en 1–3 peticiones es lo que evita el 429. El pico de audio en RAM es ~7 MB por bloque de 4000 caracteres, aceptable en 512 MB.
- `_ejecutar()` igual: `timeout=180`, `capture_output=True`.
- **La estructura de** `partes` (lista de tuplas con closures) se mantiene: es ligera en RAM.
- `convertir_a_m4a()` sin cambios: ya era eficiente.

Lo que deberías medir después de aplicar esto

Antes de dar por bueno el resultado en Render 512 MB:

1. **RSS base** de tu app arrancada, sin generar nada (`ps -o rss= -p <pid>`). Si supera 300 MB, optimizar `video.py` no alcanzará.

2. **RSS pico** durante una generación de, digamos, 5 escenas. Si el pico queda por debajo de 480 MB, tienes margen.
3. **Cuota de Gemini**: confirma que la narración sigue saliendo en 1–3 peticiones, no una por escena.

Si el RSS base ya es alto, los siguientes sospechosos son el SDK `google-genai`, Pillow importado arriba y el framework web. Ahí ya no es cuestión de `video.py`.

si optimizamos la resolución se gana algo.

Optimizar resolución: qué se gana y qué se pierde

Respuesta corta: **sí, se gana RAM, pero menos de lo que la gente suele creer**. Y en tu caso concreto hay un matiz importante que puede hacer que **no valga la pena**.

1. Dónde afecta realmente la resolución

La resolución (1280×720 → 960×540, por ejemplo) impacta en **cuatro sitios distintos**, con magnitudes muy diferentes:

Componente	Impacto de bajar resolución	Magnitud real
Imagen Pillow en RAM	Proporcional al área	🟢 Notable
Buffer de x264	Proporcional al área + lookahead	🟡 Moderado
fade/overlay en ffmpeg	Proporcional al frame	🟢 Pequeño
Audio, narración, concat	Ninguno	⚪ Cero

2. Cálculo concreto: 1280×720 → 960×540

Área de 1280×720 = **921,600 px**
Área de 960×540 = **518,400 px**
Reducción = **44% menos píxeles**

a) Imagen RGB en Pillow

- 1280×720×3 bytes = **2.64 MB por imagen**
- 960×540×3 bytes = **1.48 MB por imagen**
- **Ahorro: ~1.2 MB por imagen viva**

⚠️ **Aquí está el matiz:** en la Opción B, **solo hay una imagen viva a la vez** y se libera con `del imagen` antes de la siguiente. Así que este ahorro es de **~1.2 MB en el pico**, no acumulativo. **Es ruido comparado con 512 MB.**

b) Buffer de x264

x264 mantiene:

- Frame actual + frames de referencia + lookahead
- Con `rc-lookahead=2` y `sync-lookahead=0`, son pocos frames
- Por frame: YUV420 ≈ 1.5 bytes/píxel
  - 1280×720 → **1.38 MB/frame**
  - 960×540 → **0.78 MB/frame**
- Con ~5 frames en vuelo: **~7 MB → ~4 MB**, ahorro **~3 MB**

También hay tablas internas de x264 (motion estimation, etc.) que escalan con la resolución, pero son de orden MB, no decenas.

Ahorro total en x264: ~5–10 MB.

c) ffmpeg `fade` filter

El filtro `fade` mantiene un buffer temporal del frame. Igual que x264, escala con el área:

- **Ahorro: ~1–2 MB**

d) Lo que no cambia

- Audio PCM/WAV/AAC: idéntico
- Gemini TTS: idéntico
- Concat final: usa `-c copy`, no decodifica
- Fuentes Pillow (`lru_cache=8`): idéntico

3. Ahorro total realista

Escenario	Ahorro RAM al bajar a 960×540
Opción B actual (una imagen viva, x264 serial)	**~5–10 MB en el pico**
Si tuvieras TODAS las imágenes en RAM a la vez (20 escenas)	~25 MB acumulado
Si tuvieras x264 corriendo en paralelo (N workers)	~5–10 MB por worker

Con tu código actual, bajar resolución te ahorra ~5–10 MB. Eso es **1–2% de tu presupuesto de 512 MB**. No es la palanca que necesitas.

4. Lo que sí ganas con resolución reducida (además de RAM)

Bajar a 960×540 también te da:

- **CPU más bajo**: x264 codifica 44% menos píxeles → cada segmento tarda menos. En Render free (CPU compartida y lenta), esto puede ser **el beneficio más tangible**: videos que antes tardaban 3 min pasan a 2 min.
- **Archivos MP4 más pequeños**: útil si los guardas en disco efímero o los sirves por red.
- **Menos ancho de banda** al reproducirlos.

Pero **no te ahorra cuota de Gemini** ni acelera la narración (que suele ser el cuello de botella real con TTS).

5. Cuándo sí vale la pena bajar resolución

- Si tu RSS base ya está cerca del límite y **cada MB cuenta** (por ejemplo, RSS base 400 MB, sin margen).
- Si Render te mata por **CPU time**, no por RAM (los planes free tienen CPU muy limitada).
- Si el video se va a ver en móvil o embebido, 960×540 o incluso 854×480 se ve perfectamente.
- Si necesitas generar videos **más largos** y el tiempo de ffmpeg es el problema.

6. Cuándo no vale la pena

- Si tu problema real es el **RSS base** (Gemini SDK + FastAPI + Pillow cargados). Ahí 10 MB no te salvan.
- Si el problema es la **cuota de Gemini TTS** (bajar resolución no cambia nada).
- Si el video se proyecta en pantalla grande o se sube a YouTube (720p es el mínimo aceptable hoy).

7. Cómo hacerlo si decides bajar resolución

Opción mínima: solo cambiar constantes

```python
ANCHO, ALTO = 960, 540
```

Todos los márgenes y tamaños de fuente están calibrados para 1280×720. Con 960×540:

- Los textos pueden quedar desproporcionados (fuente 40 sobre caja más pequeña).
- **MARGEN = 80** pasa a ser ~15% del ancho en vez de ~6%. Habría que bajarlo a ~50.
- Los tamaños de fuente en `_texto_ajustado` (40, 36, 32, 28, 24) siguen funcionando porque el algoritmo elige el mayor que cabe.

Opción robusta: mantener 1280×720 y escalar en ffmpeg

Dibujas a 1280×720 (como ahora) y añades `-vf scale=960:540` en `_segmento()`:

```python
"-vf", f"{fundidos},scale=960:540",
```

- **No ahorras RAM de Pillow** (la imagen sigue siendo 1280×720).
- **Sí ahorras RAM de x264** (codifica a 960×540).
- **Mantienes la composición visual** intacta.
- **Añades un poco de CPU** por el escalado.

Esta es la opción más limpia si solo quieres el ahorro de x264 sin romper el diseño visual.

Opción intermedia recomendada

Si vas a bajar resolución de verdad, baja también:

```python
ANCHO, ALTO = 960, 540
MARGEN = 60
FPS = 20          # 25 → 20, aún más ahorro de CPU y buffer
```

Y sube tamaños mínimos de fuente en `_texto_ajustado` de (40, 36, 32, 28, 24) a (36, 32, 28, 24, 20) para que el texto no se vea gigante.

8. Veredicto para tu caso

Con la Opción B ya aplicada:

- **Ahorro por bajar a 960×540: ~5–10 MB** → insignificante frente a 512 MB.
- **Ahorro por bajar FPS de 25 a 15: ~2–3 MB** → igual de insignificante.
- **Ahorro por bajar a 854×480: ~8–12 MB** → sigue siendo ruido.

No es la palanca que necesitas. Las palancas reales siguen siendo:

1. **RSS base de tu app** (medirlo primero).
2. `google-genai` **SDK** (puede ser 80–150 MB él solo).
3. **Pillow importado a nivel de módulo** (muévelo dentro de las funciones).
4. **Fuentes en** `lru_cache` (ya mitigado con maxsize=8).
5. **x264 preset/lookahead** (ya mitigado con ultrafast + rc-lookahead=2).

9. Si aún así quieres exprimir más

Además de resolución, estas dan más ahorro por esfuerzo:

Cambio	Ahorro	Esfuerzo
Importar Pillow dentro de las funciones que dibujan	~10–20 MB de RSS base	Bajo
`MALLOC_TRIM_THRESHOLD_=100000` en el entorno	Devuelve RAM al SO tras cada segmento	Trivial
`PYTHONMALLOC=malloc` en el entorno	Mejor liberación de memoria	Trivial
Un solo worker (`--workers 1`)	Evita duplicar RSS	Trivial
`gc.collect()` tras cada segmento	Fuerza liberación inmediata	Bajo
No cargar Gemini SDK hasta `_cliente_gemini()`	Ya lo haces ✅	—

Ejemplo de `gc.collect()` en el bucle de segmentos:

```python
import gc
for (nombre, dibujar, _, duracion_seg), audio in zip(partes, audios):
    imagen = carpeta / f"{nombre}.png"
    dibujar(imagen)
    segmento = carpeta / f"{nombre}.mp4"
    _segmento(ffmpeg, imagen, audio, duracion_seg, segmento)
    segmentos.append(segmento)
    del imagen
    gc.collect()   # <- fuerza liberación antes de la siguiente escena
```

Esto no baja el RSS base, pero **evita que se acumule basura entre escenas** y hace que el pico sea más estable.

Resumen en una frase

Bajar la resolución te ahorra ~5–10 MB en tu configuración actual (Opción B), lo cual es **1–2% de tu presupuesto**. Útil como ajuste complementario, pero **no resuelve el problema** si tu RSS base ya está alto. Mide primero el RSS base, optimiza imports y SDK, y solo después plantéate tocar resolución.

con está opción como queda el codigo. "-vf", f"{fundidos},scale=960:540",

`video.py` con escalado en ffmpeg (mantiene Pillow a 1280×720)

Solo cambia **una línea** en `_segmento()`. Te dejo el archivo completo para que no haya dudas, con el cambio marcado como `CAMBIO 5`.

```python
"""Video MP4 narrado a partir de un Guion de Clase.

Responsable en el equipo Kairos G10: Dario Higuera Moreno (No Code Developer).

Cada escena del guion se convierte en una diapositiva (Pillow) con su narración
en voz femenina o masculina (ver exports/narracion.py). La clase abre con un
saludo, enlaza las escenas con transiciones y cierra con una despedida, como lo
haría un docente. Si no hay voz disponible (o VIDEO_TTS=off), cada escena se
muestra en silencio durante su `duracion_seg`. ffmpeg une todo en un MP4
H.264/AAC con volumen normalizado y fundidos suaves entre diapositivas: se usa
el ffmpeg del sistema y, si no hay, el que trae el paquete `imageio-ffmpeg`.
"""
```

```python
from __future__ import annotations

import functools
import re
import shutil
import subprocess
import tempfile
from pathlib import Path

from PIL import Image, ImageDraw, ImageFont

from nuevamente.config import settings
from nuevamente.exports.narracion import narrar
from nuevamente.schemas.formatos import GuionDeClaseContenido

# Se incluye en la clave de caché de la API: al cambiar cómo se genera el video,
# los videos ya guardados se regeneran.
VERSION = 5  # 5: escalado a 960x540 en ffmpeg para aligerar x264

ANCHO, ALTO = 1280, 720          # resolución de dibujo (Pillow): se mantiene
ANCHO_SALIDA, ALTO_SALIDA = 960, 540   # resolución del MP4 final (la reduce ffmpeg)
MARGEN = 80
FPS = 25
DURACION_PORTADA_SEG = 4
DURACION_CIERRE_SEG = 3
PAUSA_FINAL_SEG = 0.9  # respiro tras la narración de cada diapositiva
FUNDIDO_SEG = 0.35

# Paleta de la interfaz web (ver web/styles.css).
FONDO = (255, 248, 241)
TEXTO = (45, 36, 64)
TEXTO_SUAVE = (110, 100, 128)
PRIMARIO = (108, 78, 245)
CORAL = (255, 122, 89)
BLANCO = (255, 255, 255)

_FUENTES_NEGRITA = [
    "/System/Library/Fonts/Supplemental/Arial Rounded Bold.ttf",
    "/System/Library/Fonts/Supplemental/Arial Bold.ttf",
    "/usr/share/fonts/truetype/dejavu/DejaVuSans-Bold.ttf",
    "/usr/share/fonts/dejavu/DejaVuSans-Bold.ttf",
]
_FUENTES_NORMAL = [
    "/System/Library/Fonts/Supplemental/Arial.ttf",
    "/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf",
    "/usr/share/fonts/dejavu/DejaVuSans.ttf",
]


class VideoError(Exception):
    """No se pudo generar el video (ffmpeg ausente o falló)."""


# ---------- Herramientas externas ----------

def _ffmpeg() -> str:
    ruta = shutil.which("ffmpeg")
    if ruta:
        return ruta
    try:
        import imageio_ffmpeg

        return imageio_ffmpeg.get_ffmpeg_exe()
    except Exception as exc:  # paquete ausente o binario no disponible para la plataforma
        raise VideoError("No se encontró ffmpeg. Instálalo o instala el paquete imageio-ffmpeg.") from exc


def _ejecutar(comando: list[str]) -> None:
    try:
        subprocess.run(comando, check=True, capture_output=True, timeout=180)
    except subprocess.CalledProcessError as exc:
        detalle = exc.stderr.decode("utf-8", errors="replace")[-600:]
```

```
        raise VideoError(f"ffmpeg falló: {detalle}") from exc
    except subprocess.TimeoutExpired as exc:
        raise VideoError("ffmpeg tardó demasiado en generar el video.") from exc


def _duracion(ffmpeg: str, archivo: Path) -> float:
    """Duración en segundos de un audio, leída de la salida de `ffmpeg -i`."""
    salida = subprocess.run([ffmpeg, "-i", str(archivo)], capture_output=True, text=True).stderr
    m = re.search(r"Duration: (\d+):(\d+):(\d+(?:\.\d+)?)", salida)
    if not m:
        raise VideoError(f"No se pudo leer la duración de {archivo.name}")
    horas, minutos, segundos = m.groups()
    return int(horas) * 3600 + int(minutos) * 60 + float(segundos)


# ---------- Guion hablado ----------

_TRANSICIONES = [
    "Sigamos.",
    "Ahora veamos lo siguiente.",
    "Continuemos.",
    "Pasemos al siguiente punto.",
]


def _texto_hablado(indice: int, total: int, narracion: str) -> str:
    """Narración de la escena con la transición que diría un docente."""
    if indice == 1:
        return narracion
    if indice == total:
        return f"Y para terminar. {narracion}"
    return f"{_TRANSICIONES[(indice - 2) % len(_TRANSICIONES)]} {narracion}"


# ---------- Diapositivas ----------

# CAMBIO 1: lru_cache de fuentes 32 -> 8.
@functools.lru_cache(maxsize=8)
def _fuente(tamano: int, negrita: bool = False) -> ImageFont.FreeTypeFont:
    for ruta in _FUENTES_NEGRITA if negrita else _FUENTES_NORMAL:
        if Path(ruta).exists():
            return ImageFont.truetype(ruta, tamano)
    return ImageFont.load_default(size=tamano)


def _degradado(img: Image.Image, caja: tuple[int, int, int, int], desde, hasta) -> None:
    x0, y0, x1, y1 = caja
    dibujo = ImageDraw.Draw(img)
    for x in range(x0, x1):
        t = (x - x0) / max(1, x1 - x0 - 1)
        color = tuple(round(a + (b - a) * t) for a, b in zip(desde, hasta))
        dibujo.line([(x, y0), (x, y1)], fill=color)


def _envolver(dibujo: ImageDraw.ImageDraw, texto: str, fuente, ancho_max: int) -> list[str]:
    lineas: list[str] = []
    actual = ""
    for palabra in texto.split():
        prueba = f"{actual} {palabra}".strip()
        if dibujo.textlength(prueba, font=fuente) <= ancho_max:
            actual = prueba
        else:
            if actual:
                lineas.append(actual)
            actual = palabra
    if actual:
        lineas.append(actual)
    return lineas


def _texto_ajustado(dibujo, texto: str, ancho: int, alto: int, tamanos, negrita=False):
    """Elige el mayor tamaño de letra con el que el texto cabe en la caja."""
```

```
    # CAMBIO 2: variables inicializadas antes del bucle.
    fuente = _fuente(tamanos[-1], negrita)
    lineas = _envolver(dibujo, texto, fuente, ancho)
    interlineado = round(tamanos[-1] * 1.4)
    for tamano in tamanos:
        fuente = _fuente(tamano, negrita)
        lineas = _envolver(dibujo, texto, fuente, ancho)
        interlineado = round(tamano * 1.4)
        if len(lineas) * interlineado <= alto:
            return fuente, lineas, interlineado
    maximo = max(1, alto // interlineado)
    lineas = lineas[:maximo]
    lineas[-1] = lineas[-1].rstrip(".,;: ") + "…"
    return fuente, lineas, interlineado


def _portada(titulo: str, total_escenas: int, minutos: int, destino: Path) -> None:
    img = Image.new("RGB", (ANCHO, ALTO), FONDO)
    _degradado(img, (0, 0, ANCHO, ALTO), PRIMARIO, CORAL)
    dibujo = ImageDraw.Draw(img)
    dibujo.text((MARGEN, 110), "GUION DE CLASE", font=_fuente(30, True), fill=BLANCO)
    fuente, lineas, inter = _texto_ajustado(dibujo, titulo, ANCHO - 2 * MARGEN, 300, (72, 64, 56, 48, 40), Tru
e)
    y = 170
    for linea in lineas:
        dibujo.text((MARGEN, y), linea, font=fuente, fill=BLANCO)
        y += inter
    dibujo.text(
        (MARGEN, y + 24),
        f"{total_escenas} escenas · aprox. {minutos} min",
        font=_fuente(32),
        fill=BLANCO,
    )
    dibujo.text((MARGEN, ALTO - 90), "Kairos", font=_fuente(34, True), fill=BLANCO)
    img.save(destino)


def _cierre(total_escenas: int, destino: Path) -> None:
    img = Image.new("RGB", (ANCHO, ALTO), FONDO)
    _degradado(img, (0, 0, ANCHO, ALTO), PRIMARIO, CORAL)
    dibujo = ImageDraw.Draw(img)
    dibujo.text((MARGEN, 230), "¡Gracias!", font=_fuente(96, True), fill=BLANCO)
    dibujo.text((MARGEN, 360), f"Repasaste {total_escenas} escenas de esta clase.", font=_fuente(34), fill=BLA
NCO)
    dibujo.text((MARGEN, ALTO - 90), "Kairos", font=_fuente(34, True), fill=BLANCO)
    img.save(destino)


def _diapositiva(indice: int, total: int, titulo: str, narracion: str, destino: Path) -> None:
    img = Image.new("RGB", (ANCHO, ALTO), FONDO)
    _degradado(img, (0, 0, ANCHO, 130), PRIMARIO, CORAL)
    dibujo = ImageDraw.Draw(img)

    dibujo.text((MARGEN, 26), f"ESCENA {indice} DE {total}", font=_fuente(22, True), fill=BLANCO)
    fuente_titulo, lineas_titulo, _ = _texto_ajustado(dibujo, titulo, ANCHO - 2 * MARGEN, 50, (40, 34, 28), Tr
ue)
    dibujo.text((MARGEN, 60), lineas_titulo[0], font=fuente_titulo, fill=BLANCO)

    fuente, lineas, inter = _texto_ajustado(
        dibujo, narracion, ANCHO - 2 * MARGEN, ALTO - 130 - 150, (40, 36, 32, 28, 24)
    )
    y = 175
    for linea in lineas:
        dibujo.text((MARGEN, y), linea, font=fuente, fill=TEXTO)
        y += inter

    base = ALTO - 70
    dibujo.rounded_rectangle((MARGEN, base, ANCHO - MARGEN, base + 10), radius=5, fill=(240, 226, 211))
    avance = MARGEN + round((ANCHO - 2 * MARGEN) * indice / total)
    dibujo.rounded_rectangle((MARGEN, base, avance, base + 10), radius=5, fill=PRIMARIO)
    dibujo.text((MARGEN, base + 22), "Kairos", font=_fuente(22, True), fill=TEXTO_SUAVE)
    img.save(destino)
```

```
# ---------- Ensamblado ----------

def _segmento(ffmpeg: str, imagen: Path, audio: Path | None, duracion_seg: float, destino: Path) -> None:
    """Una diapositiva con su narración (o silencio), con fundido de entrada y salida."""
    total = _duracion(ffmpeg, audio) + PAUSA_FINAL_SEG if audio is not None else float(duracion_seg)
    fundidos = f"fade=t=in:st=0:d={FUNDIDO_SEG},fade=t=out:st={max(0.0, total - FUNDIDO_SEG):.2f}:d={FUNDIDO_S
EG}"
    entrada_audio = ["-i", str(audio)] if audio is not None else ["-f", "lavfi", "-i", "anullsrc=r=44100:cl=st
ereo"]
    filtro_audio = "loudnorm=I=-16:TP=-1.5:LRA=11,apad" if audio is not None else "anull"
    _ejecutar([
        ffmpeg, "-y", "-loop", "1", "-framerate", str(FPS), "-i", str(imagen), *entrada_audio,
        # CAMBIO 5: el fundido se aplica sobre la imagen de dibujo y luego se escala
        # a la resolución de salida. Así x264 sólo codifica 960x540 (menos RAM en su
        # buffer interno y menos CPU por segmento), pero Pillow sigue dibujando a
        # 1280x720: la composición visual no cambia, el texto queda igual de nítido
        # y no hay que recalibrar márgenes ni tamaños de fuente.
        "-vf", f"{fundidos},scale={ANCHO_SALIDA}:{ALTO_SALIDA}",
        "-af", filtro_audio,
        # CAMBIO 3: preset ultrafast + lookahead mínimo.
        "-c:v", "libx264", "-tune", "stillimage", "-preset", "ultrafast",
        "-threads", str(settings.video_hilos),
        "-x264-params", "rc-lookahead=2:sync-lookahead=0",
        "-pix_fmt", "yuv420p", "-r", str(FPS),
        "-c:a", "aac", "-ar", "44100", "-ac", "2", "-b:a", "160k",
        "-t", f"{total:.2f}", str(destino),
    ])


def _titulo_escena(apoyo_visual: str, indice: int) -> str:
    titulo = re.sub(r"^\s*diapositiva\s*:\s*", "", apoyo_visual, flags=re.IGNORECASE).strip()
    if not titulo or titulo == "Documento completo":
        return f"Parte {indice}"
    return titulo


def generar_video(guion: GuionDeClaseContenido, titulo: str, destino: Path, voz: str = "femenina") -> str:
    """Renderiza el guion completo a un MP4 en `destino` y devuelve el motor de voz que
    lo narró ("gemini" o "sistema").

    `voz` es "femenina" o "masculina"; si no hay ninguna voz de ese tipo disponible,
    el video sale sin narración.
    """
    ffmpeg = _ffmpeg()
    destino = Path(destino)
    destino.parent.mkdir(parents=True, exist_ok=True)
    total = len(guion.escenas)

    with tempfile.TemporaryDirectory(prefix="kairos-video-") as tmp:
        carpeta = Path(tmp)
        partes: list[tuple[str, object, str, float]] = []

        def agregar(nombre: str, dibujar, texto: str, duracion_seg: float) -> None:
            partes.append((nombre, dibujar, texto, duracion_seg))

        agregar(
            "000",
            lambda ruta: _portada(titulo, total, guion.duracion_total_min, ruta),
            f"Hola, te doy la bienvenida. En esta clase vamos a repasar: {titulo}. ¡Comencemos!",
            DURACION_PORTADA_SEG,
        )
        for i, escena in enumerate(guion.escenas, start=1):
            titulo_escena = _titulo_escena(escena.apoyo_visual, i)
            agregar(
                f"{i:03d}",
                lambda ruta, i=i, t=titulo_escena, n=escena.narracion: _diapositiva(i, total, t, n, ruta),
                _texto_hablado(i, total, escena.narracion),
                escena.duracion_seg,
            )
        agregar(
```

```
            "999",
            lambda ruta: _cierre(total, ruta),
            "Eso es todo por esta clase. Gracias por tu atención, y nos vemos en la próxima.",
            DURACION_CIERRE_SEG,
        )

        # CAMBIO 4-B: narrar TODO de una vez (respeta la cuota gratuita de Gemini),
        # pero dibujar y segmentar escena por escena.
        audios, motor = narrar([(texto, carpeta / nombre) for nombre, _, texto, _ in partes], voz)

        segmentos: list[Path] = []
        for (nombre, dibujar, _, duracion_seg), audio in zip(partes, audios):
            imagen = carpeta / f"{nombre}.png"
            dibujar(imagen)
            segmento = carpeta / f"{nombre}.mp4"
            _segmento(ffmpeg, imagen, audio, duracion_seg, segmento)
            segmentos.append(segmento)
            del imagen

        lista = carpeta / "segmentos.txt"
        lista.write_text("".join(f"file '{s.name}'\n" for s in segmentos), encoding="utf-8")
        temporal = carpeta / "video.mp4"
        _ejecutar([
            ffmpeg, "-y", "-f", "concat", "-safe", "0", "-i", str(lista),
            "-c", "copy", "-movflags", "+faststart", str(temporal),
        ])
        shutil.move(str(temporal), destino)
    return motor


def convertir_a_m4a(texto: str, voz: str, destino: Path) -> str:
    """Narra `texto` con la voz pedida, lo guarda como M4A (reproducible en el navegador)
    y devuelve el motor que lo narró ("gemini" o "sistema")."""
    ffmpeg = _ffmpeg()
    destino = Path(destino)
    destino.parent.mkdir(parents=True, exist_ok=True)
    with tempfile.TemporaryDirectory(prefix="kairos-voz-") as tmp:
        (audio,), motor = narrar([(texto, Path(tmp) / "muestra")], voz)
        if audio is None:
            raise VideoError(f"No hay una voz {voz} disponible.")
        temporal = Path(tmp) / "muestra.m4a"
        _ejecutar([
            ffmpeg, "-y", "-i", str(audio), "-af", "loudnorm=I=-16:TP=-1.5:LRA=11",
            "-c:a", "aac", "-ar", "44100", "-b:a", "160k", str(temporal),
        ])
        shutil.move(str(temporal), destino)
    return motor
```

Qué cambió exactamente (y qué no)

Cambió

1. Constantes nuevas al inicio

```
python

ANCHO, ALTO = 1280, 720          # resolución de dibujo (Pillow): se mantiene
ANCHO_SALIDA, ALTO_SALIDA = 960, 540   # resolución del MP4 final
```

Ahora `ANCHO` / `ALTO` siguen siendo el lienzo de Pillow (1280×720) y `ANCHO_SALIDA` / `ALTO_SALIDA` es lo que produce ffmpeg.

2. Una línea en `_segmento()`

```
python
```

```
# antes:
"-vf", fundidos,

# ahora:
"-vf", f"{fundidos},scale={ANCHO_SALIDA}:{ALTO_SALIDA}",
```

El orden importa: **fade primero, scale después**. El fade se aplica sobre la imagen de 1280×720 (donde están los colores exactos), y el escalado a 960×540 lo hace ffmpeg con su interpolador. Si lo invirtieras (`scale` primero, `fade` después), el fade se aplicaría a la imagen ya reducida, con los mismos colores pero menor resolución — es equivalente visualmente, pero aplicar fade en el lienzo grande es más barato porque el filtro ya está corriendo ahí.

3. `VERSION = 5`
Fuerza regeneración de videos ya cacheados por la API.

No cambió

- **Pillow sigue dibujando a 1280×720.** Los márgenes, tamaños de fuente y composición visual quedan idénticos. No hay que recalibrar nada.
- `_portada`, `_cierre`, `_diapositiva` intactos.
- `narrar()` sigue llamándose una vez con todos los textos (respeta la cuota Gemini).
- **El bucle de segmentos** sigue liberando la imagen con `del imagen` antes de la siguiente.
- `convertir_a_m4a()` intacto.
- `_ejecutar`, `_ffmpeg`, `_duracion` intactos.

Qué ganas con este cambio

Aspecto	Antes (solo Opción B)	Ahora (Opción B + scale)
RAM del buffer de x264	~7 MB	~4 MB
CPU de codificación x264	100%	~56% (44% menos píxeles)
Tamaño del MP4 final	100%	~60–70%
Composición visual	1280×720	Idéntica (Pillow no cambia)
Nitidez del texto tras escalar	—	Ligeramente menor a 960×540

Ahorro de RAM real: ~3–5 MB en el pico. Igual que antes: pequeño pero real. **El beneficio más tangible es CPU**: en Render free la CPU compartida es el recurso más escaso, y codificar 44% menos píxeles puede recortar el tiempo total de generación de forma notable.

Consideraciones importantes

1. Nitidez del texto

Escalar de 1280×720 a 960×540 con el escalador por defecto de ffmpeg (bicúbico) suaviza ligeramente los bordes del texto. Para diapositivas con texto mediano/grande se ve bien. Si tu texto es pequeño o fino, considera:

```
python

"-vf", f"{fundidos},scale={ANCHO_SALIDA}:{ALTO_SALIDA}:flags=lanczos",
```

`lanczos` da mejor nitidez a costa de un poco más de CPU. Sigue siendo más barato que codificar a 1280×720.

2. Relación de aspecto

1280×720 y 960×540 tienen exactamente la misma relación (16:9). No hay deformación ni barras negras. Si algún día cambias a una resolución de salida con otra relación, tendrías que añadir `pad` o `crop` en el filtro.

3. Si más adelante quieres bajar aún más

Puedes tocar solo `ANCHO_SALIDA` / `ALTO_SALIDA` sin tocar `ANCHO` / `ALTO`. Por ejemplo:

```python
ANCHO_SALIDA, ALTO_SALIDA = 854, 480   # 480p
```

Y el resto del código no se entera. Esa es la ventaja de esta estrategia: **separas la resolución de dibujo de la resolución de salida**.

4. Si quieres volver a 720p

Simplemente:

```python
ANCHO_SALIDA, ALTO_SALIDA = 1280, 720
```

El filtro `scale=1280:720` sobre una imagen ya de 1280×720 es un no-op barato (ffmpeg lo detecta). Vuelves al comportamiento anterior sin tocar más código.

Veredicto

Este cambio es **limpio, reversible y no rompe nada del diseño visual**. Te da:

- ~3–5 MB de ahorro de RAM en el pico
- ~40% menos CPU de codificación
- MP4 finales ~30–40% más pequeños

No es la palanca que soluciona 512 MB por sí sola, pero **sumado a los cambios 1, 3 y 4-B ya tienes un conjunto coherente**: fuentes acotadas, x264 ligero, procesamiento por escena y salida reducida. Si aún así te quedas corto en Render, el siguiente sospechoso es el RSS base (Gemini SDK, Pillow, framework), no este archivo.

genero el video pero sin la voz.

Diagnóstico: el video sale pero sin voz

Si el video se genera correctamente pero **no tiene narración**, el problema está en la cadena de audio, no en ffmpeg ni en Pillow. Vamos por partes para localizarlo.

1. Primero: ¿qué motor de voz se está usando?

En `generar_video()` devuelves `motor` como string. **Verifica qué valor tiene** en tu ejecución:

```python
motor = generar_video(guion, titulo, destino, voz)
print(f"Motor de voz usado: {motor}")   # ¿"gemini" o "sistema"?
```

- Si devuelve `"sistema"` → Gemini falló y cayó al respaldo. En Render Linux **no hay** `say` **(macOS)**, así que depende de `espeak-ng`.
- Si devuelve `"gemini"` pero no hay voz → el audio se generó pero `_segmento()` no lo está usando.

2. Causas más probables (en orden)

🔴 Causa 1: `espeak-ng` no está instalado en Render

En Linux, `sintetizar()` busca:

```python
def _espeak() -> str | None:
    return shutil.which("espeak-ng") or shutil.which("espeak")
```

Si **ninguno** de los dos existe, `sintetizar()` devuelve `None` → la escena sale **en silencio**. Esto es lo más común en Render porque:

- El contenedor base de Render **no incluye** `espeak-ng`.
- Tu `Dockerfile` / `render.yaml` probablemente no lo instala.

Solución: añade `espeak-ng` a tu imagen. En `Dockerfile`:

```dockerfile
RUN apt-get update && apt-get install -y \
    ffmpeg \
    espeak-ng \
    && rm -rf /var/lib/apt/lists/*
```

O en `render.yaml` si usas buildpacks, necesitas un `Dockerfile` personalizado. Render free **no permite instalar paquetes del sistema fuera de Docker**.

⚠️ **Nota:** `espeak-ng` suena robótico. Si Render free es tu único entorno, considera si vale la pena o si prefieres exigir Gemini.

🔴 Causa 2: `VIDEO_TTS=off` en las variables de entorno

En `narrar()`:

```python
if settings.video_tts == "off" or not textos:
    return [None] * len(textos), "sistema"
```

Y en `sintetizar()`:

```python
if settings.video_tts == "off" or tipo not in TIPOS_VOZ:
    return None
```

Si en Render tienes `VIDEO_TTS=off` configurado, **todo sale en silencio a propósito**.

Verifica en Render: Settings → Environment → busca `VIDEO_TTS`. Si está en `off`, cámbialo a `on` o elimínalo.

🟠 Causa 3: Gemini TTS falla y el respaldo tampoco funciona

Si `motor_voz()` devuelve `"gemini"` pero la llamada falla (sin API key, sin cuota, sin red), `narrar()` cae al respaldo:

```python
except Exception as exc:
    logger.warning("Gemini TTS no disponible, se narra con la voz del sistema: %s", exc)
    disponibilidad.pausar(SERVICIO_TTS, str(exc))
return [sintetizar(texto, tipo, destino) for texto, destino in textos], "sistema"
```

Pero si `espeak-ng` **no está instalado** (Causa 1), `sintetizar()` devuelve `None` para cada texto → **silencio total**.

Cómo confirmarlo: revisa los logs de Render buscando:

```
text

Gemini TTS no disponible, se narra con la voz del sistema: ...
```

🟡 Causa 4: API key de Gemini ausente o inválida

`gemini_configurado()`:

```
python

return settings.video_voces == "gemini" and bool(settings.gemini_api_key or settings.gemini_api_keys)
```

Si `VIDEO_VOCES=gemini` pero **no hay** `GEMINI_API_KEY`, `motor_voz()` devuelve `"sistema"` → depende de `espeak-ng` (Causa 1 otra vez).

Verifica en Render: que exista la variable `GEMINI_API_KEY` (o `GEMINI_API_KEYS`) con una key válida.

🟡 Causa 5: `_segmento()` recibe `audio=None` pero no lo notas

En `_segmento()`:

```
python

entrada_audio = ["-i", str(audio)] if audio is not None else ["-f", "lavfi", "-i", "anullsrc=r=44100:cl=stere
o"]
filtro_audio = "loudnorm=I=-16:TP=-1.5:LRA=11,apad" if audio is not None else "anull"
```

Si `audio` es `None`, se genera **silencio a propósito** (`anullsrc` + `anull`). El video sale bien, con pista de audio muda.

Esto es lo que está pasando con casi total seguridad. `narrar()` está devolviendo `[None, None, None, ...]`.

🟢 Causa 6: El audio se genera pero ffmpeg no lo mezcla

Menos probable, pero posible: si el archivo de audio no tiene la duración esperada, `_duracion()` podría fallar, o el `-t` final corta antes de que empiece el audio.

Descartable si `audio is not None` y ffmpeg no lanza error.

3. Cómo diagnosticarlo paso a paso

Paso 1: Añade logs temporales en `narrar()`

Edita `narracion.py` (temporalmente) para ver qué está pasando:

```
python

def narrar(textos, tipo):
    from nuevamente.llm import disponibilidad

    logger.warning("narrar() llamado con %d textos, tipo=%s, tts=%s, gemini_configurado=%s",
                   len(textos), tipo, settings.video_tts, gemini_configurado())

    if settings.video_tts == "off" or not textos:
        logger.warning("Saliendo sin narración: video_tts=%s o textos vacíos", settings.video_tts)
        return [None] * len(textos), "sistema"

    if motor_voz() == "gemini":
```

```
        try:
            audios = _narrar_gemini(textos, tipo)
            logger.warning("Gemini OK: %d audios", len(audios))
            disponibilidad.reanudar(SERVICIO_TTS)
            return audios, "gemini"
        except Exception as exc:
            logger.warning("Gemini falló: %s", exc)
            disponibilidad.pausar(SERVICIO_TTS, str(exc))
    else:
        logger.warning("motor_voz() devolvió 'sistema', no se intenta Gemini")

    audios = [sintetizar(texto, tipo, destino) for texto, destino in textos]
    logger.warning("Sistema: %d de %d audios generados", sum(a is not None for a in audios), len(audios))
    return audios, "sistema"
```

Despliega y mira los logs de Render. Verás exactamente dónde se corta.

Paso 2: Verifica variables de entorno en Render

En el dashboard de Render → tu servicio → Environment:

Variable	Valor esperado
`VIDEO_TTS`	`on` (o ausente)
`VIDEO_VOCES`	`gemini` (o ausente si quieres default)
`GEMINI_API_KEY`	Una key válida de Google AI Studio

Paso 3: Verifica que `espeak-ng` esté instalado (si usas respaldo)

Añade un endpoint temporal o un log:

```
python

import shutil
logger.warning("espeak-ng: %s | espeak: %s | ffmpeg: %s",
               shutil.which("espeak-ng"), shutil.which("espeak"), shutil.which("ffmpeg"))
```

Si los tres primeros son `None`, ese es el problema.

Paso 4: Prueba la narración aislada

Llama directamente a `convertir_a_m4a()` desde un endpoint de prueba:

```
python

from nuevamente.exports.video import convertir_a_m4a

@router.get("/debug/voz")
def debug_voz():
    try:
        motor = convertir_a_m4a("Hola, esta es una prueba de voz.", "femenina", Path("/tmp/prueba.m4a"))
        return {"ok": True, "motor": motor, "archivo": "/tmp/prueba.m4a"}
    except Exception as e:
        return {"ok": False, "error": str(e)}
```

Si esto falla, el problema está en `narracion.py`. Si funciona pero `generar_video()` no narra, el problema está en cómo `generar_video()` llama a `narrar()`.

4. La causa más probable en Render free

Sospecho esta combinación:

1. En Render Linux **no hay** `say` **de macOS**.

2. `espeak-ng` **no está instalado** en tu contenedor.
3. **Gemini TTS no está configurado** (falta `GEMINI_API_KEY` o `VIDEO_VOCES` no es `gemini`).
4. O Gemini **sí está configurado pero falla** (cuota, red, timeout) y cae al respaldo, que tampoco existe.

Resultado: `narrar()` devuelve `[None] * N` → `_segmento()` genera silencio → **video mudo**.

5. Soluciones según el caso

Si quieres usar Gemini TTS (recomendado)

1. **Configura las variables en Render:**

```
text

VIDEO_TTS=on
VIDEO_VOCES=gemini
GEMINI_API_KEY=tu_key_aqui
```

2. **Verifica la cuota** del plan gratuito de Gemini. El módulo `narracion.py` ya agrupa todo en 1–3 peticiones, pero si la key no tiene TTS habilitado, falla igual.
3. **Aumenta el timeout** si Render es lento:

```
python

# en settings
llm_timeout_s = 60   # o más
```

4. **Revisa los logs** buscando `"Gemini TTS no disponible"`.

Si quieres usar voz del sistema en Render

Necesitas **Dockerfile personalizado** porque Render free no permite `apt-get` en buildpacks:

```
dockerfile

FROM python:3.11-slim

RUN apt-get update && apt-get install -y \
    ffmpeg \
    espeak-ng \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .
CMD ["uvicorn", "nuevamente.main:app", "--host", "0.0.0.0", "--port", "10000"]
```

Y en `render.yaml`:

```
yaml

services:
  - type: web
    name: kairos
    runtime: docker
    dockerfilePath: ./Dockerfile
    plan: free
```

⚠️ **Pero cuidado:** `espeak-ng` suena robótico y poco profesional para una clase. Yo priorizaría arreglar Gemini.

Si aceptas video mudo como fallback

El código **ya hace esto por diseño**. Si no hay voz, el video sale en silencio pero con las diapositivas bien. Si eso es aceptable para ti en Render free (mientras en tu máquina local sí tienes Gemini), entonces **no hay bug**: es el comportamiento esperado.

6. Checklist rápido

Marca lo que aplique en tu caso:

- ☐ ¿`VIDEO_TTS` está en `on` o ausente en Render?
- ☐ ¿`VIDEO_VOCES` está en `gemini` o ausente?
- ☐ ¿`GEMINI_API_KEY` existe y es válida?
- ☐ ¿Los logs de Render muestran `"Gemini TTS no disponible"`?
- ☐ ¿`espeak-ng` está instalado en el contenedor? (`shutil.which("espeak-ng")`)
- ☐ ¿`generar_video()` devuelve `"gemini"` o `"sistema"`?
- ☐ ¿`convertir_a_m4a("prueba", "femenina", ...)` genera audio?

7. Resumen en una frase

El video sale mudo porque `narrar()` está devolviendo `[None, None, ...]`. Las dos causas casi seguras en Render free son: (a) `espeak-ng` no está instalado y Gemini falla, o (b) `VIDEO_TTS=off` / falta `GEMINI_API_KEY`. Añade los logs del Paso 1, mira los logs de Render, y en 2 minutos sabrás cuál es.

funciono. pero la letra está un poco grande. como se mejora en el codigo.

Ajustar el tamaño de la letra en las diapositivas

El tamaño de la letra lo decide `_texto_ajustado()`, que prueba una lista de tamaños **de mayor a menor** y se queda con el primero que cabe. Si la letra se ve grande, es porque:

1. Los tamaños máximos de esas listas son muy altos.
2. La caja disponible (`alto`) es generosa.
3. No hay un multiplicador global que escale todo.

Aquí tienes las tres formas de arreglarlo, de menos a más invasiva.

Opción 1 (recomendada): un factor de escala global

Añade una constante y multiplícala en `_texto_ajustado()`. Así ajustas todo el video con un solo número.

```python
# Nueva constante junto a las demás, cerca de ANCHO/ALTO:
ESCALA_TEXTO = 0.85   # 1.0 = como está ahora; 0.85 = 15% más pequeño
```

Y en `_texto_ajustado()`, aplica el factor a la lista de tamaños antes de probarlos:

```python
def _texto_ajustado(dibujo, texto: str, ancho: int, alto: int, tamanos, negrita=False):
    """Elige el mayor tamaño de letra con el que el texto cabe en la caja."""
    # CAMBIO: aplicar el factor de escala global a los tamaños candidatos.
    tamanos = [max(10, round(t * ESCALA_TEXTO)) for t in tamanos]

    fuente = _fuente(tamanos[-1], negrita)
    lineas = _envolver(dibujo, texto, fuente, ancho)
    interlineado = round(tamanos[-1] * 1.4)
    for tamano in tamanos:
```

```
        fuente = _fuente(tamano, negrita)
        lineas = _envolver(dibujo, texto, fuente, ancho)
        interlineado = round(tamano * 1.4)
        if len(lineas) * interlineado <= alto:
            return fuente, lineas, interlineado
    maximo = max(1, alto // interlineado)
    lineas = lineas[:maximo]
    lineas[-1] = lineas[-1].rstrip(".,;: ") + "…"
    return fuente, lineas, interlineado
```

Cómo calibrarlo: empieza con `0.85`. Si sigue grande, baja a `0.80` o `0.75`. Si queda pequeño, sube a `0.90`.

Ventaja: un solo número controla **todo** el texto (títulos, narración, portada, cierre). Reversible en un segundo.

Desventaja: no diferencia entre títulos y cuerpo. Si quieres títulos grandes y cuerpo pequeño, necesitas la Opción 3.

Opción 2: bajar los tamaños máximos en cada llamada

Si prefieres control fino por zona, edita directamente las listas de tamaños en cada función. **El orden importa: de mayor a menor.**

En `_diapositiva()` — texto de la narración (el que se ve grande)

```python
# antes:
fuente, lineas, inter = _texto_ajustado(
    dibujo, narracion, ANCHO - 2 * MARGEN, ALTO - 130 - 150, (40, 36, 32, 28, 24)
)

# después (bajando los topes):
fuente, lineas, inter = _texto_ajustado(
    dibujo, narracion, ANCHO - 2 * MARGEN, ALTO - 130 - 150, (34, 30, 27, 24, 21, 18)
)
```

En `_diapositiva()` — título de la escena (en la franja superior)

```python
# antes:
fuente_titulo, lineas_titulo, _ = _texto_ajustado(
    dibujo, titulo, ANCHO - 2 * MARGEN, 50, (40, 34, 28), True
)

# después:
fuente_titulo, lineas_titulo, _ = _texto_ajustado(
    dibujo, titulo, ANCHO - 2 * MARGEN, 50, (34, 30, 26), True
)
```

En `_portada()` — título grande de la portada

```python
# antes:
fuente, lineas, inter = _texto_ajustado(
    dibujo, titulo, ANCHO - 2 * MARGEN, 300, (72, 64, 56, 48, 40), True
)

# después:
fuente, lineas, inter = _texto_ajustado(
    dibujo, titulo, ANCHO - 2 * MARGEN, 300, (60, 54, 48, 42, 36, 30), True
)
```

En `_portada()` — subtítulo "N escenas · aprox. M min"

```python
# antes:
font=_fuente(32),

# después:
font=_fuente(26),
```

En `_cierre()` — "¡Gracias!" y subtítulo

```python
# antes:
dibujo.text((MARGEN, 230), "¡Gracias!", font=_fuente(96, True), fill=BLANCO)
dibujo.text((MARGEN, 360), f"Repasaste {total_escenas} escenas de esta clase.", font=_fuente(34), fill=BLANCO)

# después:
dibujo.text((MARGEN, 230), "¡Gracias!", font=_fuente(80, True), fill=BLANCO)
dibujo.text((MARGEN, 360), f"Repasaste {total_escenas} escenas de esta clase.", font=_fuente(28), fill=BLANCO)
```

En `_diapositiva()` — etiqueta "ESCENA X DE Y" y pie "Kairos"

```python
# antes:
dibujo.text((MARGEN, 26), f"ESCENA {indice} DE {total}", font=_fuente(22, True), fill=BLANCO)
dibujo.text((MARGEN, base + 22), "Kairos", font=_fuente(22, True), fill=TEXTO_SUAVE)

# después:
dibujo.text((MARGEN, 26), f"ESCENA {indice} DE {total}", font=_fuente(18, True), fill=BLANCO)
dibujo.text((MARGEN, base + 22), "Kairos", font=_fuente(18, True), fill=TEXTO_SUAVE)
```

Ventaja: control total por zona. Puedes hacer el título más grande y el cuerpo más pequeño sin afectar al resto.

Desventaja: más sitios donde tocar. Si mañana cambias de opinión, tienes que repasar 6 listas.

Opción 3 (mi recomendación): factor global + un ajuste fino

Combina lo mejor de las dos. Añade **dos constantes** al inicio:

```python
ESCALA_TEXTO_TITULO = 0.90   # títulos, encabezados, portada
ESCALA_TEXTO_CUERPO = 0.82   # narración, textos largos
```

Y crea un pequeño helper:

```python
def _escalar(tamanos, factor):
    """Aplica un factor a una tupla/lista de tamaños candidatos."""
    return [max(10, round(t * factor)) for t in tamanos]
```

Luego úsalo en cada llamada:

```python
# _portada — título
fuente, lineas, inter = _texto_ajustado(
    dibujo, titulo, ANCHO - 2 * MARGEN, 300, _escalar((72, 64, 56, 48, 40), ESCALA_TEXTO_TITULO), True
)

# _diapositiva — título de escena
```

```
fuente_titulo, lineas_titulo, _ = _texto_ajustado(
    dibujo, titulo, ANCHO - 2 * MARGEN, 50, _escalar((40, 34, 28), ESCALA_TEXTO_TITULO), True
)

# _diapositiva — narración (el cuerpo)
fuente, lineas, inter = _texto_ajustado(
    dibujo, narracion, ANCHO - 2 * MARGEN, ALTO - 130 - 150,
    _escalar((40, 36, 32, 28, 24), ESCALA_TEXTO_CUERPO)
)
```

Ventaja: dos números controlan todo, pero con proporciones distintas para títulos y cuerpo. Es lo que usaría yo.

¿Cuál elegir?

Situación	Opción
"Todo está un poco grande, bájalo parejo"	**Opción 1** (`ESCALA_TEXTO = 0.85`)
"Solo el texto de la narración está grande, el resto bien"	**Opción 2**, solo la lista de `_diapositiva()` narración
"Quiero títulos grandes y cuerpo más pequeño"	**Opción 3**
"No sé qué quiero, voy a probar"	**Opción 1**, y mueves el número hasta que te guste

Nota sobre el interlineado

`interlineado = round(tamano * 1.4)` es el espacio entre líneas. Si bajas mucho la letra, 1.4 queda bien. Si subes la letra y quieres más aire, súbelo a 1.5:

```
python

interlineado = round(tamano * 1.5)
```

Pero si cambias esto, afecta a `_texto_ajustado` (que decide si cabe) y a la posición de cada línea (`y += inter`). Es consistente en todo el archivo, así que es seguro.

Lo que no recomiendo

- **No** cambies `ANCHO` / `ALTO` para reducir la letra. Eso reduce todo (márgenes, posiciones, barra de avance) y tendrías que recalibrar todo el layout.
- **No** uses `scale` en ffmpeg para reducir el texto. Ya lo estás usando para 960×540. Si además quieres texto más pequeño, hazlo en Pillow, no en ffmpeg.
- **No** toques `MARGEN` pensando que cambia el tamaño de la letra. MARGEN es el espacio lateral; afecta a cuánto texto cabe por línea, no al tamaño.

Cómo probar rápido

1. Aplica la **Opción 1** con `ESCALA_TEXTO = 0.85`.
2. Genera un video corto (2–3 escenas) y míralo.
3. Ajusta el número: `0.80` si sigue grande, `0.90` si quedó pequeño.
4. Cuando lo tengas, súbelo a Render.

Con `0.85` como punto de partida, la mayoría de guiones se ven bien. Si tienes textos muy largos que ocupan muchas líneas, prueba `0.80`.

que linea está ESCALA_TEXTO = 0.85.

Dónde va exactamente `ESCALA_TEXTO = 0.85`

No existe todavía en tu código: **hay que añadirla**. Va en el bloque de constantes del inicio del archivo, junto a `ANCHO`, `ALTO`, `MARGEN`, etc.

Ubicación exacta

En `video.py`, busca este bloque:

```python
# Se incluye en la clave de caché de la API: al cambiar cómo se genera el video,
# los videos ya guardados se regeneran.
VERSION = 5  # 5: escalado a 960x540 en ffmpeg para aligerar x264

ANCHO, ALTO = 1280, 720          # resolución de dibujo (Pillow): se mantiene
ANCHO_SALIDA, ALTO_SALIDA = 960, 540   # resolución del MP4 final (la reduce ffmpeg)
MARGEN = 80
FPS = 25
DURACION_PORTADA_SEG = 4
DURACION_CIERRE_SEG = 3
PAUSA_FINAL_SEG = 0.9  # respiro tras la narración de cada diapositiva
FUNDIDO_SEG = 0.35
```

Añade la nueva constante justo después de `FUNDIDO_SEG` (o antes, da igual, pero que quede dentro de ese bloque):

```python
# Se incluye en la clave de caché de la API: al cambiar cómo se genera el video,
# los videos ya guardados se regeneran.
VERSION = 6  # 6: factor de escala global para el tamaño del texto

ANCHO, ALTO = 1280, 720          # resolución de dibujo (Pillow): se mantiene
ANCHO_SALIDA, ALTO_SALIDA = 960, 540   # resolución del MP4 final (la reduce ffmpeg)
MARGEN = 80
FPS = 25
DURACION_PORTADA_SEG = 4
DURACION_CIERRE_SEG = 3
PAUSA_FINAL_SEG = 0.9  # respiro tras la narración de cada diapositiva
FUNDIDO_SEG = 0.35

ESCALA_TEXTO = 0.85   # <-- AQUÍ. 1.0 = como está ahora; 0.85 = 15% más pequeño
```

Segundo cambio: usarla en `_texto_ajustado()`

La constante por sí sola no hace nada. Hay que multiplicarla dentro de `_texto_ajustado()`, en la parte donde se prueba cada tamaño. Busca esta función:

```python
def _texto_ajustado(dibujo, texto: str, ancho: int, alto: int, tamanos, negrita=False):
    """Elige el mayor tamaño de letra con el que el texto cabe en la caja."""
    # CAMBIO 2: variables inicializadas antes del bucle.
    fuente = _fuente(tamanos[-1], negrita)
    lineas = _envolver(dibujo, texto, fuente, ancho)
    interlineado = round(tamanos[-1] * 1.4)
    for tamano in tamanos:
        fuente = _fuente(tamano, negrita)
        lineas = _envolver(dibujo, texto, fuente, ancho)
        interlineado = round(tamano * 1.4)
        if len(lineas) * interlineado <= alto:
            return fuente, lineas, interlineado
    maximo = max(1, alto // interlineado)
    lineas = lineas[:maximo]
    lineas[-1] = lineas[-1].rstrip(".,;: ") + "…"
    return fuente, lineas, interlineado
```

Y déjala así (solo se añade **una línea** al principio del cuerpo):

```python
def _texto_ajustado(dibujo, texto: str, ancho: int, alto: int, tamanos, negrita=False):
    """Elige el mayor tamaño de letra con el que el texto cabe en la caja."""
    # CAMBIO: aplicar el factor de escala global a los tamaños candidatos.
    tamanos = [max(10, round(t * ESCALA_TEXTO)) for t in tamanos]

    # CAMBIO 2: variables inicializadas antes del bucle.
    fuente = _fuente(tamanos[-1], negrita)
    lineas = _envolver(dibujo, texto, fuente, ancho)
    interlineado = round(tamanos[-1] * 1.4)
    for tamano in tamanos:
        fuente = _fuente(tamano, negrita)
        lineas = _envolver(dibujo, texto, fuente, ancho)
        interlineado = round(tamano * 1.4)
        if len(lineas) * interlineado <= alto:
            return fuente, lineas, interlineado
    maximo = max(1, alto // interlineado)
    lineas = lineas[:maximo]
    lineas[-1] = lineas[-1].rstrip(".,;: ") + "…"
    return fuente, lineas, interlineado
```

⚠️ Advertencia importante

Esa línea afecta a **todo** el texto que pasa por `_texto_ajustado()`:

- Título de la portada
- Título de cada escena
- Narración de cada escena

Pero NO afecta a los textos que van directos con `_fuente(...)`:

- `"GUION DE CLASE"` en portada → `_fuente(30, True)`
- `"N escenas · aprox. M min"` en portada → `_fuente(32)`
- `"Kairos"` en portada y cierre → `_fuente(34, True)`
- `"¡Gracias!"` en cierre → `_fuente(96, True)`
- `"Repasaste N escenas..."` en cierre → `_fuente(34)`
- `"ESCENA X DE Y"` en cada diapositiva → `_fuente(22, True)`
- `"Kairos"` del pie → `_fuente(22, True)`

Si también quieres reducir esos, hay que bajarlos manualmente (o envolverlos en una función helper). Por eso en el mensaje anterior te di tres opciones: la 1 es la más simple, la 3 es la más completa.

Cómo verificar que funciona

1. Añade `ESCALA_TEXTO = 0.85` al bloque de constantes.
2. Añade la línea `tamanos = [max(10, round(t * ESCALA_TEXTO)) for t in tamanos]` dentro de `_texto_ajustado()`.
3. Sube `VERSION` a `6` para forzar regeneración.
4. Genera un video corto y compáralo.

Si el texto de la narración sigue grande pero los títulos ya se ven bien, prueba `0.78` o `0.75`. Si queda demasiado pequeño, `0.90`.

Resumen de las dos ediciones

Dónde	Qué añadir
Bloque de constantes al inicio (después de `FUNDIDO_SEG`)	`ESCALA_TEXTO = 0.85`
Primera línea del cuerpo de `_texto_ajustado()`	`tamanos = [max(10, round(t * ESCALA_TEXTO)) for t in tamanos]`
Constante `VERSION`	Subir de `5` a `6`

Genera el codigo completo. me queda mas sencillo para pegar.

`video.py` completo con `ESCALA_TEXTO`

Aquí tienes el archivo entero listo para pegar. Los cambios nuevos están marcados como `CAMBIO 6`.

python

```python
"""Video MP4 narrado a partir de un Guion de Clase.

Responsable en el equipo Kairos G10: Dario Higuera Moreno (No Code Developer).

Cada escena del guion se convierte en una diapositiva (Pillow) con su narración
en voz femenina o masculina (ver exports/narracion.py). La clase abre con un
saludo, enlaza las escenas con transiciones y cierra con una despedida, como lo
haría un docente. Si no hay voz disponible (o VIDEO_TTS=off), cada escena se
muestra en silencio durante su `duracion_seg`. ffmpeg une todo en un MP4
H.264/AAC con volumen normalizado y fundidos suaves entre diapositivas: se usa
el ffmpeg del sistema y, si no hay, el que trae el paquete `imageio-ffmpeg`.
"""
from __future__ import annotations

import functools
import re
import shutil
import subprocess
import tempfile
from pathlib import Path

from PIL import Image, ImageDraw, ImageFont

from nuevamente.config import settings
from nuevamente.exports.narracion import narrar
from nuevamente.schemas.formatos import GuionDeClaseContenido

# Se incluye en la clave de caché de la API: al cambiar cómo se genera el video,
# los videos ya guardados se regeneran.
VERSION = 6  # 6: factor de escala global para el tamaño del texto

ANCHO, ALTO = 1280, 720          # resolución de dibujo (Pillow): se mantiene
ANCHO_SALIDA, ALTO_SALIDA = 960, 540   # resolución del MP4 final (la reduce ffmpeg)
MARGEN = 80
FPS = 25
DURACION_PORTADA_SEG = 4
DURACION_CIERRE_SEG = 3
PAUSA_FINAL_SEG = 0.9  # respiro tras la narración de cada diapositiva
FUNDIDO_SEG = 0.35

# CAMBIO 6: factor de escala global del texto que pasa por _texto_ajustado().
# 1.0 = como estaba antes; 0.85 = 15% más pequeño. Ajusta este número para
# agrandar o reducir títulos y cuerpo sin tocar cada lista de tamaños.
ESCALA_TEXTO = 0.85

# Paleta de la interfaz web (ver web/styles.css).
FONDO = (255, 248, 241)
TEXTO = (45, 36, 64)
TEXTO_SUAVE = (110, 100, 128)
PRIMARIO = (108, 78, 245)
CORAL = (255, 122, 89)
```

```python
BLANCO = (255, 255, 255)

_FUENTES_NEGRITA = [
    "/System/Library/Fonts/Supplemental/Arial Rounded Bold.ttf",
    "/System/Library/Fonts/Supplemental/Arial Bold.ttf",
    "/usr/share/fonts/truetype/dejavu/DejaVuSans-Bold.ttf",
    "/usr/share/fonts/dejavu/DejaVuSans-Bold.ttf",
]
_FUENTES_NORMAL = [
    "/System/Library/Fonts/Supplemental/Arial.ttf",
    "/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf",
    "/usr/share/fonts/dejavu/DejaVuSans.ttf",
]


class VideoError(Exception):
    """No se pudo generar el video (ffmpeg ausente o falló)."""


# ---------- Herramientas externas ----------

def _ffmpeg() -> str:
    ruta = shutil.which("ffmpeg")
    if ruta:
        return ruta
    try:
        import imageio_ffmpeg

        return imageio_ffmpeg.get_ffmpeg_exe()
    except Exception as exc:  # paquete ausente o binario no disponible para la plataforma
        raise VideoError("No se encontró ffmpeg. Instálalo o instala el paquete imageio-ffmpeg.") from exc


def _ejecutar(comando: list[str]) -> None:
    try:
        subprocess.run(comando, check=True, capture_output=True, timeout=180)
    except subprocess.CalledProcessError as exc:
        detalle = exc.stderr.decode("utf-8", errors="replace")[-600:]
        raise VideoError(f"ffmpeg falló: {detalle}") from exc
    except subprocess.TimeoutExpired as exc:
        raise VideoError("ffmpeg tardó demasiado en generar el video.") from exc


def _duracion(ffmpeg: str, archivo: Path) -> float:
    """Duración en segundos de un audio, leída de la salida de `ffmpeg -i`."""
    salida = subprocess.run([ffmpeg, "-i", str(archivo)], capture_output=True, text=True).stderr
    m = re.search(r"Duration: (\d+):(\d+):(\d+(?:\.\d+)?)", salida)
    if not m:
        raise VideoError(f"No se pudo leer la duración de {archivo.name}")
    horas, minutos, segundos = m.groups()
    return int(horas) * 3600 + int(minutos) * 60 + float(segundos)


# ---------- Guion hablado ----------

_TRANSICIONES = [
    "Sigamos.",
    "Ahora veamos lo siguiente.",
    "Continuemos.",
    "Pasemos al siguiente punto.",
]


def _texto_hablado(indice: int, total: int, narracion: str) -> str:
    """Narración de la escena con la transición que diría un docente."""
    if indice == 1:
        return narracion
    if indice == total:
        return f"Y para terminar. {narracion}"
    return f"{_TRANSICIONES[(indice - 2) % len(_TRANSICIONES)]} {narracion}"
```

```python
# ---------- Diapositivas ----------

# CAMBIO 1: lru_cache de fuentes 32 -> 8.
@functools.lru_cache(maxsize=8)
def _fuente(tamano: int, negrita: bool = False) -> ImageFont.FreeTypeFont:
    for ruta in _FUENTES_NEGRITA if negrita else _FUENTES_NORMAL:
        if Path(ruta).exists():
            return ImageFont.truetype(ruta, tamano)
    return ImageFont.load_default(size=tamano)


def _degradado(img: Image.Image, caja: tuple[int, int, int, int], desde, hasta) -> None:
    x0, y0, x1, y1 = caja
    dibujo = ImageDraw.Draw(img)
    for x in range(x0, x1):
        t = (x - x0) / max(1, x1 - x0 - 1)
        color = tuple(round(a + (b - a) * t) for a, b in zip(desde, hasta))
        dibujo.line([(x, y0), (x, y1)], fill=color)


def _envolver(dibujo: ImageDraw.ImageDraw, texto: str, fuente, ancho_max: int) -> list[str]:
    lineas: list[str] = []
    actual = ""
    for palabra in texto.split():
        prueba = f"{actual} {palabra}".strip()
        if dibujo.textlength(prueba, font=fuente) <= ancho_max:
            actual = prueba
        else:
            if actual:
                lineas.append(actual)
            actual = palabra
    if actual:
        lineas.append(actual)
    return lineas


def _texto_ajustado(dibujo, texto: str, ancho: int, alto: int, tamanos, negrita=False):
    """Elige el mayor tamaño de letra con el que el texto cabe en la caja."""
    # CAMBIO 6: aplicar el factor de escala global a los tamaños candidatos.
    tamanos = [max(10, round(t * ESCALA_TEXTO)) for t in tamanos]

    # CAMBIO 2: variables inicializadas antes del bucle.
    fuente = _fuente(tamanos[-1], negrita)
    lineas = _envolver(dibujo, texto, fuente, ancho)
    interlineado = round(tamanos[-1] * 1.4)
    for tamano in tamanos:
        fuente = _fuente(tamano, negrita)
        lineas = _envolver(dibujo, texto, fuente, ancho)
        interlineado = round(tamano * 1.4)
        if len(lineas) * interlineado <= alto:
            return fuente, lineas, interlineado
    maximo = max(1, alto // interlineado)
    lineas = lineas[:maximo]
    lineas[-1] = lineas[-1].rstrip(".,;: ") + "…"
    return fuente, lineas, interlineado


def _portada(titulo: str, total_escenas: int, minutos: int, destino: Path) -> None:
    img = Image.new("RGB", (ANCHO, ALTO), FONDO)
    _degradado(img, (0, 0, ANCHO, ALTO), PRIMARIO, CORAL)
    dibujo = ImageDraw.Draw(img)
    dibujo.text((MARGEN, 110), "GUION DE CLASE", font=_fuente(30, True), fill=BLANCO)
    fuente, lineas, inter = _texto_ajustado(dibujo, titulo, ANCHO - 2 * MARGEN, 300, (72, 64, 56, 48, 40), Tru
e)
    y = 170
    for linea in lineas:
        dibujo.text((MARGEN, y), linea, font=fuente, fill=BLANCO)
        y += inter
    dibujo.text(
        (MARGEN, y + 24),
        f"{total_escenas} escenas · aprox. {minutos} min",
        font=_fuente(32),
```

```
        fill=BLANCO,
    )
    dibujo.text((MARGEN, ALTO - 90), "Kairos", font=_fuente(34, True), fill=BLANCO)
    img.save(destino)


def _cierre(total_escenas: int, destino: Path) -> None:
    img = Image.new("RGB", (ANCHO, ALTO), FONDO)
    _degradado(img, (0, 0, ANCHO, ALTO), PRIMARIO, CORAL)
    dibujo = ImageDraw.Draw(img)
    dibujo.text((MARGEN, 230), "¡Gracias!", font=_fuente(96, True), fill=BLANCO)
    dibujo.text((MARGEN, 360), f"Repasaste {total_escenas} escenas de esta clase.", font=_fuente(34), fill=BLA
NCO)
    dibujo.text((MARGEN, ALTO - 90), "Kairos", font=_fuente(34, True), fill=BLANCO)
    img.save(destino)


def _diapositiva(indice: int, total: int, titulo: str, narracion: str, destino: Path) -> None:
    img = Image.new("RGB", (ANCHO, ALTO), FONDO)
    _degradado(img, (0, 0, ANCHO, 130), PRIMARIO, CORAL)
    dibujo = ImageDraw.Draw(img)

    dibujo.text((MARGEN, 26), f"ESCENA {indice} DE {total}", font=_fuente(22, True), fill=BLANCO)
    fuente_titulo, lineas_titulo, _ = _texto_ajustado(dibujo, titulo, ANCHO - 2 * MARGEN, 50, (40, 34, 28), Tr
ue)
    dibujo.text((MARGEN, 60), lineas_titulo[0], font=fuente_titulo, fill=BLANCO)

    fuente, lineas, inter = _texto_ajustado(
        dibujo, narracion, ANCHO - 2 * MARGEN, ALTO - 130 - 150, (40, 36, 32, 28, 24)
    )
    y = 175
    for linea in lineas:
        dibujo.text((MARGEN, y), linea, font=fuente, fill=TEXTO)
        y += inter

    base = ALTO - 70
    dibujo.rounded_rectangle((MARGEN, base, ANCHO - MARGEN, base + 10), radius=5, fill=(240, 226, 211))
    avance = MARGEN + round((ANCHO - 2 * MARGEN) * indice / total)
    dibujo.rounded_rectangle((MARGEN, base, avance, base + 10), radius=5, fill=PRIMARIO)
    dibujo.text((MARGEN, base + 22), "Kairos", font=_fuente(22, True), fill=TEXTO_SUAVE)
    img.save(destino)


# ---------- Ensamblado ----------

def _segmento(ffmpeg: str, imagen: Path, audio: Path | None, duracion_seg: float, destino: Path) -> None:
    """Una diapositiva con su narración (o silencio), con fundido de entrada y salida."""
    total = _duracion(ffmpeg, audio) + PAUSA_FINAL_SEG if audio is not None else float(duracion_seg)
    fundidos = f"fade=t=in:st=0:d={FUNDIDO_SEG},fade=t=out:st={max(0.0, total - FUNDIDO_SEG):.2f}:d={FUNDIDO_S
EG}"
    entrada_audio = ["-i", str(audio)] if audio is not None else ["-f", "lavfi", "-i", "anullsrc=r=44100:cl=st
ereo"]
    filtro_audio = "loudnorm=I=-16:TP=-1.5:LRA=11,apad" if audio is not None else "anull"
    _ejecutar([
        ffmpeg, "-y", "-loop", "1", "-framerate", str(FPS), "-i", str(imagen), *entrada_audio,
        # CAMBIO 5: fundido sobre la imagen de dibujo y luego escalado a la salida.
        "-vf", f"{fundidos},scale={ANCHO_SALIDA}:{ALTO_SALIDA}",
        "-af", filtro_audio,
        # CAMBIO 3: preset ultrafast + lookahead mínimo.
        "-c:v", "libx264", "-tune", "stillimage", "-preset", "ultrafast",
        "-threads", str(settings.video_hilos),
        "-x264-params", "rc-lookahead=2:sync-lookahead=0",
        "-pix_fmt", "yuv420p", "-r", str(FPS),
        "-c:a", "aac", "-ar", "44100", "-ac", "2", "-b:a", "160k",
        "-t", f"{total:.2f}", str(destino),
    ])


def _titulo_escena(apoyo_visual: str, indice: int) -> str:
    titulo = re.sub(r"^\s*diapositiva\s*:\s*", "", apoyo_visual, flags=re.IGNORECASE).strip()
    if not titulo or titulo == "Documento completo":
        return f"Parte {indice}"
```

```
    return titulo


def generar_video(guion: GuionDeClaseContenido, titulo: str, destino: Path, voz: str = "femenina") -> str:
    """Renderiza el guion completo a un MP4 en `destino` y devuelve el motor de voz que
    lo narró ("gemini" o "sistema").

    `voz` es "femenina" o "masculina"; si no hay ninguna voz de ese tipo disponible,
    el video sale sin narración.
    """
    ffmpeg = _ffmpeg()
    destino = Path(destino)
    destino.parent.mkdir(parents=True, exist_ok=True)
    total = len(guion.escenas)

    with tempfile.TemporaryDirectory(prefix="kairos-video-") as tmp:
        carpeta = Path(tmp)
        partes: list[tuple[str, object, str, float]] = []

        def agregar(nombre: str, dibujar, texto: str, duracion_seg: float) -> None:
            partes.append((nombre, dibujar, texto, duracion_seg))

        agregar(
            "000",
            lambda ruta: _portada(titulo, total, guion.duracion_total_min, ruta),
            f"Hola, te doy la bienvenida. En esta clase vamos a repasar: {titulo}. ¡Comencemos!",
            DURACION_PORTADA_SEG,
        )
        for i, escena in enumerate(guion.escenas, start=1):
            titulo_escena = _titulo_escena(escena.apoyo_visual, i)
            agregar(
                f"{i:03d}",
                lambda ruta, i=i, t=titulo_escena, n=escena.narracion: _diapositiva(i, total, t, n, ruta),
                _texto_hablado(i, total, escena.narracion),
                escena.duracion_seg,
            )
        agregar(
            "999",
            lambda ruta: _cierre(total, ruta),
            "Eso es todo por esta clase. Gracias por tu atención, y nos vemos en la próxima.",
            DURACION_CIERRE_SEG,
        )

        # CAMBIO 4-B: narrar TODO de una vez (respeta la cuota gratuita de Gemini),
        # pero dibujar y segmentar escena por escena.
        audios, motor = narrar([(texto, carpeta / nombre) for nombre, _, texto, _ in partes], voz)

        segmentos: list[Path] = []
        for (nombre, dibujar, _, duracion_seg), audio in zip(partes, audios):
            imagen = carpeta / f"{nombre}.png"
            dibujar(imagen)
            segmento = carpeta / f"{nombre}.mp4"
            _segmento(ffmpeg, imagen, audio, duracion_seg, segmento)
            segmentos.append(segmento)
            del imagen

        lista = carpeta / "segmentos.txt"
        lista.write_text("".join(f"file '{s.name}'\n" for s in segmentos), encoding="utf-8")
        temporal = carpeta / "video.mp4"
        _ejecutar([
            ffmpeg, "-y", "-f", "concat", "-safe", "0", "-i", str(lista),
            "-c", "copy", "-movflags", "+faststart", str(temporal),
        ])
        shutil.move(str(temporal), destino)
    return motor


def convertir_a_m4a(texto: str, voz: str, destino: Path) -> str:
    """Narra `texto` con la voz pedida, lo guarda como M4A (reproducible en el navegador)
    y devuelve el motor que lo narró ("gemini" o "sistema")."""
    ffmpeg = _ffmpeg()
    destino = Path(destino)
```

```
    destino.parent.mkdir(parents=True, exist_ok=True)
    with tempfile.TemporaryDirectory(prefix="kairos-voz-") as tmp:
        (audio,), motor = narrar([(texto, Path(tmp) / "muestra")], voz)
        if audio is None:
            raise VideoError(f"No hay una voz {voz} disponible.")
        temporal = Path(tmp) / "muestra.m4a"
        _ejecutar([
            ffmpeg, "-y", "-i", str(audio), "-af", "loudnorm=I=-16:TP=-1.5:LRA=11",
            "-c:a", "aac", "-ar", "44100", "-b:a", "160k", str(temporal),
        ])
        shutil.move(str(temporal), destino)
    return motor
```

Cómo ajustar el tamaño

Solo tienes que cambiar **un número**, la constante `ESCALA_TEXTO`:

Valor	Efecto
`1.00`	Tamaño original (como antes)
`0.90`	10% más pequeño
`0.85`	15% más pequeño (valor actual)
`0.80`	20% más pequeño
`0.75`	25% más pequeño
`0.70`	30% más pequeño

Genera un video de prueba, mira cómo queda, y sube o baja el número. **No olvides subir `VERSION` cada vez que cambies algo del render** para que la caché de la API regenere los videos ya guardados.

Qué textos NO afecta `ESCALA_TEXTO`

Como te comenté antes, `ESCALA_TEXTO` solo afecta al texto que pasa por `_texto_ajustado()`:

- ✅ Título de la portada
- ✅ Título de cada escena
- ✅ Narración de cada escena

Estos siguen con tamaño fijo porque van directos con `_fuente(...)`:

- ❌ `"GUION DE CLASE"` (portada, 30)
- ❌ `"N escenas · aprox. M min"` (portada, 32)
- ❌ `"Kairos"` (portada y cierre, 34)
- ❌ `"¡Gracias!"` (cierre, 96)
- ❌ `"Repasaste N escenas..."` (cierre, 34)
- ❌ `"ESCENA X DE Y"` (cada diapositiva, 22)
- ❌ `"Kairos"` del pie (22)

Si al bajar a `0.85` notas que esos textos fijos quedan desproporcionados (por ejemplo, el título de la escena se ve más chico que la etiqueta "ESCENA X DE Y"), dímelo y te preparo una versión con factor separado para títulos y cuerpo, o un helper que también escale los textos fijos.